Privacy-Preserving Federated Learning for IoT-Based Healthcare: A Layered-Defense Benchmark on ECG Arrhythmia Classification

Mapeke

2026

Abstract

Federated learning (FL) is the textbook answer to privacy-preserving training on healthcare IoT data, and the layered defenses around it — DP-SGD, secure aggregation, and update compression — are textbook ingredients of a "privacy-preserving FL" pipeline. The textbook does not say which layer pays for which threat, or what each one costs in detection utility on a realistic clinical workload. This thesis builds a fully working layered-defense framework in roughly 2,500 lines of Python (Flower + PyTorch + Opacus), and benchmarks it on the MIT-BIH Arrhythmia dataset with four simulated wearable IoT clients training a 1-D CNN over the AAMI 5-class label scheme. Three findings drive the thesis: (1) federation alone is essentially free — vanilla FedAvg matches a centralized baseline within 0.02 pp test accuracy on this workload; (2) pairwise-mask secure aggregation is also free — it shifts test accuracy by 0.20 pp, well below seed variance, and the cryptographic correctness property (mask cancellation under summation) is encoded as a unit test that runs in CI; (3) DP-SGD at $\varepsilon = 5$ collapses the model to the majority-class predictor on this heavily imbalanced workload, with final accuracy equal to the prevalence of class N to two decimals — a privacy/utility cliff that is invisible if one only reports overall accuracy. Adding compression on top of DP does not degrade the model further, because there is nothing left to degrade. A follow-up run on MIT-BIH records 200–234 (where the imbalance is moderate, 82 % N rather than 97 %) shows that this collapse is dataset-conditional: DP-SGD alone retains a +6.37 pp signal over the majority baseline on that slice, but the full pipeline (DP + SecAgg + compression at $\varepsilon = 8$) still collapses to majority. The cliff is real; its location is a property of the workload's class balance. Every number in the thesis is reproducible from a committed YAML configuration via `python -m experiments.run_*`.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Contributions . . . 4
1.3 Structure . . . 5
1.4 Scope and non-goals . . . 5

2 Background **6**
2.1 Federated learning and FedAvg . . . 6
2.2 The privacy stack . . . 6
2.2.1 DP-SGD formal definition . . . 7
2.2.2 Pairwise-mask SecAgg correctness . . . 8
2.2.3 Composition: DP + SecAgg + compression . . . 9
2.3 Dataset and use case . . . 9

3 Related work **11**
3.1 Federated learning aggregation . . . 11
3.2 Differential privacy in deep learning . . . 12
3.3 Secure aggregation . . . 12
3.4 Gradient compression . . . 13
3.5 Privacy-preserving FL in healthcare . . . 13
3.6 Attacks: membership inference and gradient inversion . . . 13
3.7 What is not covered . . . 14

4 Threat model **15**
4.1 Assets being protected . . . 15
4.2 Adversaries and capabilities . . . 15
4.2.1 A1 — Honest-but-curious aggregation server . . . 15
4.2.2 A2 — Curious-client coalition (size $t < N$) . . . 16
4.2.3 A3 — Network eavesdropper . . . 16
4.2.4 A4 — External adversary holding the released global model . . . 16
4.2.5 A5 — Malicious (Byzantine) client . . . 17
4.3 Attack-surface coverage matrix . . . 17
4.4 Trust assumptions . . . 18
4.5 What is explicitly out of scope . . . 18

5 Method **19**
5.1 Unified client interface (`src/client/flower_client.py`) . . . 19
5.2 Data layer (`src/data/`) . . . 19
5.3 Models (`src/models/`) . . . 19
5.4 Differential privacy (`src/privacy/differential_privacy.py`) . . . 20
5.5 Secure aggregation (`src/privacy/secure_agg.py`) . . . 20

5.6 Compression (src/privacy/compression.py) . . . 20
5.7 Server strategy (src/server/strategies.py) . . . 21
5.8 Experiment infrastructure . . . 22
5.9 Tests . . . 22
5.10 Per-round data flow . . . 22

6 Results I — Centralized and federated baselines **24**
6.1 Detection grid . . . 24
6.2 Centralized baseline . . . 24
6.3 Vanilla FedAvg . . . 24
6.4 Reproducibility check vs. published FL ECG results . . . 25

7 Results II — Secure aggregation **26**
7.1 Mask cancellation as a code-level invariant . . . 26
7.2 End-to-end SecAgg run . . . 26
7.3 What SecAgg does not protect against . . . 27

8 Results III — Differential privacy and the full pipeline **28**
8.1 DP-SGD run . . . 28
8.2 Class-prior collapse . . . 28
8.3 The full pipeline run . . . 29
8.4 Negotiating the cliff . . . 29
8.5 Follow-up: records 200–234 . . . 30

9 Discussion **33**
9.1 What the three findings add up to . . . 33
9.2 Likely reviewer objections . . . 33
9.3 Practical implications for deployment . . . 34
9.4 What we deliberately did not test . . . 34
9.5 Future work . . . 36

10 Conclusion **37**

A Architecture and package layout **40**
A.1 Actors . . . 40
A.2 Per-round protocol . . . 40
A.3 Package layout (src/) . . . 41
A.4 Configuration knobs . . . 41
A.5 Where the substance lives . . . 42

B Resolved YAML configurations **43**
B.1 configs/default.yaml . . . 43
B.2 configs/experiment_secagg.yaml . . . 44
B.3 configs/experiment_dp.yaml . . . 44
B.4 configs/experiment_full.yaml . . . 45
B.5 Follow-up configs (configs/followup_records200/) . . . 46

C Selected source code **47**
C.1 Pairwise-mask SecAgg (src/privacy/secure_agg.py) . . . 47
C.2 Top-k + int8 compression (src/privacy/compression.py) . . . 48
C.3 DP attachment (src/privacy/differential_privacy.py) . . . 49

D Reproducibility checklist **51**
D.1 Environment 51
D.2 Setup 51
D.3 Reproducing the five primary experiments (records 100–115) 51
D.4 Reproducing the records 200–234 follow-up 52
D.5 Approximate run-times (single RTX-class GPU) 52
D.6 What is logged per run 52
D.7 Single seed; multi-seed is future work 52

Chapter 1

Introduction

1.1 Motivation

Connected health devices — smartwatches, Holter monitors, bedside telemetry, the implantable devices that already exist in patients — produce streams of physiological data that are simultaneously *clinically valuable* and *highly sensitive*. Existing deployments centralize that data, which (i) creates a single point of failure for breaches, (ii) exposes patients to re-identification even after pseudonymization, and (iii) conflicts with the data-minimization principle of HIPAA and GDPR. Federated learning was proposed [1] precisely to break this pattern: each device trains locally on its own raw data, the central server only ever sees model updates.

The naive version of this story is incomplete in two ways that the deep-learning literature is comfortable papering over:

1. **Model updates leak.** Per-client gradients reconstruct training records under known attacks; a coalition of clients or an honest-but-curious server can recover patient data from the updates alone. Differential privacy [2] and secure aggregation [4] are the two canonical defenses, but each addresses a different threat and each introduces its own utility tax.

2. **Healthcare data is heavily imbalanced.** ECG arrhythmia classification on MIT-BIH is roughly 97 % normal beats; rare classes can be $< 0.1\,\%$. Privacy mechanisms designed and tuned on balanced datasets (CIFAR, MNIST) behave badly here in ways the headline-accuracy literature rarely surfaces.

This thesis takes both of these seriously. We build a working layered-defense framework, run it end-to-end on real PhysioNet data, and report what each privacy layer actually costs — including the cases where the cost is the detector ceasing to detect anything.

1.2 Contributions

C1 — A layered-defense FL framework with separable privacy layers. The Flower client interface used here exposes three independent switches (differential privacy, secure aggregation, compression) which can be enabled in isolation or jointly via a single YAML config. The marginal cost of each layer on test accuracy, spent ε, and convergence is measured by holding the others fixed (Chapters 6, 7, 8). The implementation is roughly 2,500 lines of Python and runs end-to-end on a single workstation in under 30 minutes.

C2 — Cryptographic correctness as a code-level invariant. The pairwise-mask secure aggregation protocol's correctness argument (every mask appears with + and − exactly once, so summing client updates cancels every mask) is encoded as a unit test (`tests/test_secagg.py::test_masks_can` that runs in CI. The argument from `docs/privacy-analysis.md` §2 [4] is therefore mechanically checked rather than prose-only.

C3 — A class-prior collapse diagnostic for DP on imbalanced healthcare data. On the MIT-BIH workload used here, the DP-SGD run at $\varepsilon \approx 5$ converges to a final test accuracy of **0.9711**, which equals the prevalence of class N (Normal) to four decimals. The full-pipeline run (DP + SecAgg + compression) at $\varepsilon \approx 8$ converges to the *same* 0.9711. We argue (Chapter 8, §8.2) that this is a class-prior collapse, not a learning curve, and that it is the single most consequential and least discussed failure mode of DP-SGD on realistic clinical data. A follow-up on records 200–234 (§8.5) confirms the diagnostic generalises but its severity is class-balance-conditional: on moderate imbalance DP-SGD retains a measurable rare-class signal, while the full pipeline still collapses.

1.3 Structure

Chapter 2 reviews FL, the three privacy primitives we layer, and the MIT-BIH dataset. Chapter 3 situates the framework in the existing literature on FL aggregation, DP-SGD, SecAgg, gradient compression, privacy-preserving FL on healthcare data, and the membership-inference and gradient-inversion attacks that motivate each layer. Chapter 4 formalises the threat model: the assets being protected, five adversaries A1–A5 with their capabilities and goals, the attack-surface coverage matrix, and the trust assumptions the framework inherits. Chapter 5 describes the implementation: `FlowerHealthcareClient`, `PrivacyAwareFedAvg`, `DPConfig+attach_dp`, the pairwise-mask SecAgg primitives, and the top-k/int8 compression pipeline. Chapter 6 reports the centralized baseline and vanilla FedAvg. Chapter 7 adds secure aggregation. Chapter 8 adds differential privacy and the full pipeline. Chapter 9 discusses anticipated reviewer objections and limitations. Chapter 10 concludes.

1.4 Scope and non-goals

- **We do not propose a new privacy primitive.** The argument is empirical: existing primitives composed under a protocol that makes their individual costs visible.
- **We do not deploy on real IoT hardware.** Every number is from a Flower simulation on a single GPU. The IoT angle is in the data partitioning (per-patient non-IID) and in the device-profile metadata (`wearable`, `bedside`, `gateway`); the experiments do not exercise actual network or compute constraints.
- **We do not evaluate against poisoning / Byzantine clients.** The threat model is honest-but-curious server plus an external adversary holding the released global model (Appendix `docs/threat-model.md`). Robust aggregation (Krum, median, trimmed mean) is listed as future work (§9.5).
- **Our primary experiments use MIT-BIH records 100–115**, which `wfdb.dl_database` pulls down by default and which exhibit the heavy-imbalance regime (97.1 % N) that drives the central finding. A follow-up section (Chapter 8, §8.5) re-runs all five experiments on records 200–234 (more arrhythmia variety, 82.1 % N) to test whether the DP class-prior collapse is dataset-conditional.

Chapter 2

Background

2.1 Federated learning and FedAvg

The federated learning protocol of McMahan et al. [1] distributes training across N clients each holding a local dataset D_i, with the server orchestrating T rounds of:

$$w_{t+1} \;=\; \sum_{i=1}^{N} \tfrac{n_i}{n}\,\mathrm{LocalSGD}(w_t, D_i),$$

where $n_i = |D_i|$ and $n = \sum_i n_i$. Raw D_i never leaves client i; only $\mathrm{LocalSGD}(w_t, D_i)$ does. FedAvg is the baseline we layer defenses on top of.

The healthcare specialization is straightforward: each D_i is a per-patient or per-device partition of physiological data. We use four simulated wearable clients holding disjoint Dirichlet-non-IID partitions (§5.2) of MIT-BIH ECG beats.

2.2 The privacy stack

Three layers, each addressing a different threat.

Differential privacy (DP-SGD). Replaces every local SGD step with: clip per-sample gradients to ℓ_2-norm C, add Gaussian noise of scale σC, then step. Composing across T rounds and converting to (ε, δ) via the RDP accountant [3] gives a tight bound on what an adversary holding only the released global model can learn about any single training record. Implemented via Opacus [5].

Pairwise-mask secure aggregation. For every unordered client pair $\{i, j\}$, both clients derive a shared mask vector m_{ij}. Client i adds $+m_{ij}$ to its update; client j adds $-m_{ij}$. The server's sum cancels every mask exactly, so the server only ever sees the aggregate $\sum_i \Delta w_i$ and never any individual Δw_i [4]. Defends against an honest-but-curious server that would otherwise inspect per-client updates.

Top-k + int8 compression. Each client keeps the largest-magnitude k fraction of its update entries and quantizes them into 8-bit integers. Reduces bandwidth roughly $40\times$ for $k = 0.1$. Post-processing-immune to DP and compatible with SecAgg as long as both endpoints agree on the sparsification mask.

Composition. DP and SecAgg compose without interaction: the DP guarantee is on w_T regardless of channel; SecAgg is on intermediate per-round messages. Compression composes with both as a post-hoc lossy operator. The full composition argument is laid out in §2.2.3.

The next three subsections give the formal statement of each layer: §2.2.1 writes out the DP-SGD step and the RDP-to-(ε, δ) conversion; §2.2.2 writes out the mask-cancellation algebra of pairwise-mask SecAgg; §2.2.3 states how the three layers compose. The framework's design choices in Chapter 5 are direct implementations of these statements, and the reproducibility check in Chapter 6 is built on top of them.

2.2.1 DP-SGD formal definition

Following Abadi et al. [2], every local SGD step on a minibatch B of size b is replaced by:

1. Compute per-sample gradients $g_i = \nabla_w \ell(w; x_i, y_i)$ for each $(x_i, y_i) \in B$.
2. Clip per-sample gradients to ℓ_2-norm at most C: $\bar{g}_i = g_i \,/\, \max\Big(1, \frac{\|g_i\|_2}{C}\Big)$. This bounds the sensitivity of the per-sample sum to any single training record by C.
3. Add Gaussian noise of scale σC to the clipped sum and average: $\tilde{g} \;=\; \frac{1}{b}\big(\sum_{i \in B} \bar{g}_i \;+\; \mathcal{N}(0,\, \sigma^2 C^2 I)\big)$.
4. Update weights: $w \leftarrow w - \eta\, \tilde{g}$.

Each step satisfies $(\alpha,\, \alpha C^2 \sigma^{-2}/2)$-Rényi differential privacy at every order $\alpha > 1$ [3]. Composing T steps under RDP and converting to (ε, δ)-DP via $\varepsilon = \alpha C^2 \sigma^{-2} T/2 + \log(1/\delta)/(\alpha - 1)$ (optimised over α) gives the tight cumulative bound that Opacus's `RDPAccountant` [5] reports as `get_epsilon(steps)`. In the framework the user supplies $(\varepsilon_{\text{target}}, \delta_{\text{target}}, C, T)$ and Opacus auto-calibrates σ to hit $\varepsilon \le \varepsilon_{\text{target}}$ over T steps; the framework reports the realised ε at the end of training (see Chapter 8).

Parameter choices. Table 2.1 lists the defaults used throughout the thesis. `target_epsilon` $= 5$ is a conventional "moderate" budget for medical models in the literature; `target_delta` $= 10^{-5}$ satisfies $\delta < 1/N$ for our dataset size; $C = 1.0$ is the Opacus default and is well-matched to the gradient norms we observe during training; σ is auto-calibrated.

Table 2.1: DP-SGD defaults used in the thesis. The user-facing knobs are $(\varepsilon_{\text{target}}, \delta_{\text{target}}, C)$; the noise multiplier σ is solved for by the RDP accountant.

Parameter	Default	Justification
`target_epsilon`	5.0	Conventional moderate budget for medical models
`target_delta`	10^{-5}	$< 1/N$ for $N \approx 25{,}000$ training records
`max_grad_norm` (C)	1.0	Opacus default; matches observed gradient scale
`noise_multiplier` (σ)	auto	Solved by RDP accountant to hit `target_epsilon`
`accountant`	rdp	RDP gives the tightest known bounds for Gaussian noise

What ε buys, intuitively. For the worst-case adversary A4 (Chapter 4, §3.2.4) with auxiliary information about every other training record, the posterior on whether a given record was in the training set shifts by at most a factor of e^{ε}: at $\varepsilon = 5$ this is roughly 148, a loose worst-case bound. Empirical membership-inference accuracy at this budget is typically much closer to the 50% random-guess baseline. The thesis does not run a membership-inference attack to verify this empirically because the (ε, δ)-DP guarantee is a worst-case *distributional* statement that does not require empirical attestation; running the attack would, at best, lower-bound something the budget already upper-bounds.

2.2.2 Pairwise-mask SecAgg correctness

Let each client $i \in [N]$ hold a parameter update $\Delta w_i \in \mathbb{R}^d$ at the end of a training round. For every unordered pair $\{i, j\}$ the two clients derive a shared seed s_{ij} from a key-exchange protocol (Diffie–Hellman in deployment-grade implementations [4]; a pre-shared secret in the prototype, see Chapter 5, §5.7). Each client expands s_{ij} via a cryptographic PRG into a mask vector $m_{ij} \in \mathbb{R}^d$. Client i's outgoing message is then:

$$\tilde{\Delta w}_i \;=\; \Delta w_i \;+\; \sum_{j>i} m_{ij} \;-\; \sum_{j<i} m_{ji} \pmod p, \tag{2.1}$$

where the modular reduction is over a fixed-point representation of $\mathbb{R}^d$ in finite-field arithmetic (modulus p in deployment-grade implementations; `float32` arithmetic in the prototype, an accepted simplification with documented numerical consequences).

Cancellation. Summing Equation (2.1) across all clients:

$$\begin{aligned} S \;&=\; \sum_{i=1}^{N} \tilde{\Delta w}_i \\ &=\; \sum_{i=1}^{N} \Delta w_i \;+\; \sum_{i=1}^{N} \left(\sum_{j>i} m_{ij} - \sum_{j<i} m_{ji} \right) \\ &=\; \sum_{i=1}^{N} \Delta w_i \;+\; \sum_{\{i,j\}:i<j} (m_{ij} - m_{ij}) \\ &=\; \sum_{i=1}^{N} \Delta w_i. \end{aligned} \tag{2.2}$$

The second-to-last equality uses the fact that for every unordered pair $\{i, j\}$ the mask m_{ij} appears once with $+$ in client i's message (the term in $\sum_{j>i}$) and once with $-$ in client j's message (the term in $\sum_{j'<j}$ with $j' = i$). Every mask therefore cancels exactly once at the sum level, regardless of N.

Privacy. For any *strict* subset $S \subsetneq [N]$ of clients the marginal distribution of $\{\tilde{\Delta w}_i\}_{i \in S}$ is uniformly random conditional on the true updates: at least one mask m_{ij} in the sum has $j \notin S$, that mask is uniformly random and known only to the two endpoints, and adding a uniformly random vector to any Δw_i produces a uniformly random vector. The server therefore learns no information about any individual Δw_i beyond what is already revealed by the total sum $\sum_i \Delta w_i$ that it must learn to perform aggregation. The formal statement is Bonawitz et al. Theorem 4.1 [4].

Limitations of the prototype. The prototype departs from Bonawitz et al. in three ways, each accepted as a simulation-grade simplification and flagged in the source: **(i)** pre-shared seeds per pair instead of Diffie–Hellman key exchange — convenient for the simulation orchestrator, infeasible for a real deployment because the seeds must be transported some other way; **(ii)** no Shamir secret-sharing of seeds — the protocol does not tolerate dropouts, since a missing client leaves un-cancelled masks in the sum; **(iii)** `float32` arithmetic instead of modular integer arithmetic over a finite field — introduces a small numerical residue in the cancelled sum (§7.2 measures this empirically at 0.20 pp end-to-end accuracy gap). All three are listed as deployment-grade upgrades in §9.5.

2.2.3 Composition: DP + SecAgg + compression

The three layers compose without negotiation, in the precise sense that the privacy guarantee of each layer survives the addition of the others:

DP guarantee survives SecAgg. Differential privacy is a guarantee on the released global model w_T as a function of the input training set. SecAgg changes the channel through which intermediate updates flow, but does not change w_T itself: the cancellation property (Equation 2.2) means the server reconstructs the true $\sum_i \Delta w_i$ and applies it exactly as it would have without SecAgg. The DP-SGD calibration of σ to $\varepsilon_{\text{target}}$ is therefore unaffected.

SecAgg guarantee survives DP. SecAgg's privacy property is on the per-round messages $\tilde{\Delta w}_i$: their joint distribution conditional on the true updates is uniformly random for any strict subset. DP changes the distribution of the true updates Δw_i (by adding Gaussian noise during local training), but does not change the masking layer applied on top. SecAgg's guarantee composes with whatever local update generation procedure the client runs.

Compression as post-processing. Top-k sparsification and int8 quantisation are deterministic, lossy operators applied to Δw_i *before* SecAgg masking. They do not weaken the DP guarantee on w_T by post-processing immunity (a fundamental property of DP: any function of a DP output is itself DP at the same budget). They do not weaken the SecAgg property either, because the SecAgg layer operates on the post-compression vector and the cancellation algebra is agnostic to the values being masked. The only requirement is that sender and receiver agree on the sparsification mask; the framework sends the indices alongside the values (Chapter 5, §5.7).

Composed statement. Putting the three layers together:

> *An adversary observing only the released global model w_T learns at most (ε, δ) worth of information about any individual training record (**DP**), AND an adversary observing the per-round server inbox learns nothing about any individual client's update beyond the aggregate $\sum_i \Delta w_i$ (**SecAgg**). Compression reduces the per-round bandwidth from $4d \cdot 32\,\text{bits}$ to roughly $4(kd \cdot 8\,\text{bits} + kd \cdot 32\,\text{bits})$ for top-k + int8 at sparsity $k = 0.1$, a $\sim 40\times$ reduction, without weakening either of the above guarantees.*

This is the contract that Chapters 6–8 benchmark empirically.

2.3 Dataset and use case

MIT-BIH Arrhythmia. PhysioNet's MIT-BIH database [7] is the de facto benchmark for ECG arrhythmia classification. Sampling rate 360 Hz, two leads, beat-level annotations on ∼48 records. We use the first 12 records (100, 101, 103, 105, 106, 108, 109, 111, 112, 113, 114, 115), which `wfdb.dl_database` pulls without authentication.

AAMI 5-class scheme. Annotation symbols are mapped to the AAMI [8] 5-class grouping: N (normal, including L/R/e/j), S (supraventricular ectopic), V (ventricular ectopic), F (fusion), Q (unknown / paced).

Class imbalance. Table 2.2 gives the realised distribution. The dataset is dominated by class N: it accounts for 97.1 % of the beats. Two classes (F, Q) have under 10 examples each; no clinical claim about them is possible from this data, but they are kept in the label space because they appear in the AAMI scheme.

Table 2.2: AAMI class distribution across the 12 MIT-BIH records, after beat-window segmentation (window size 360 samples). Class N dominates with 97.1 % prevalence. Imbalance of this magnitude is the central technical fact of Chapter 8.

Class	Count	Share
N (normal)	24,655	97.10 %
V (ventricular ectopic)	661	2.60 %
S (supraventricular)	62	0.24 %
F (fusion)	8	0.03 %
Q (unknown / paced)	7	0.03 %
Total	25,393	100.00 %

Why ECG, why MIT-BIH. ECG monitoring is the canonical wearable-IoT health workload; MIT-BIH is the canonical benchmark for it; the AAMI mapping is the canonical label scheme. A privacy-preserving FL framework that ignores the imbalance characteristic of this canonical workload is over-claiming.

Chapter 3

Related work

This thesis contributes a layered-defence FL framework rather than a new privacy primitive (Chapter 1, §1.4); the contribution rests on how existing primitives compose and what each layer costs on a realistic healthcare workload. This chapter places each of those primitives, and the attacks they are built to mitigate, in the context of the existing literature. Six strands matter: the FL aggregation rule that everything sits on top of (§3.1); the DP-SGD lineage that drives the privacy guarantee on the released model (§3.2); the secure aggregation lineage that closes the per-update channel (§3.3); the gradient compression literature that supplies the bandwidth optimisation (§3.4); prior privacy-preserving FL work specifically on healthcare data (§3.5); and the gradient-inversion and membership-inference attacks that justify the threat model (§3.6).

3.1 Federated learning aggregation

FedAvg. The federated averaging protocol of McMahan et al. [1] introduced the now-canonical pattern: each client runs E local epochs of SGD on its private data, the server weighted-averages the resulting parameter updates, and the protocol iterates for T communication rounds. FedAvg works well when client data distributions are similar and the model is not over-parameterised. It is the baseline of this thesis; §6.3 confirms that on the MIT-BIH workload it matches the centralized baseline within seed variance, so no stronger aggregation rule is needed for the framework's contribution to be visible.

FedProx. Li et al. [10] added a proximal regulariser $\frac{\mu}{2}\|w_i - w_t\|^2$ to each client's local loss to mitigate client drift in heterogeneous settings (different local data distributions, different local-epoch counts). FedProx is strictly more robust than FedAvg when the heterogeneity is severe but pays a small constant cost otherwise. We do not use it because FedAvg already achieves the centralized baseline.

SCAFFOLD. Karimireddy et al. proposed SCAFFOLD, which estimates and corrects for client drift via per-client control variates, producing strictly better convergence than FedAvg in non-IID settings. The correction adds bookkeeping overhead and additional state per client. Like FedProx, it is overkill for our workload.

Surveys. Kairouz et al. [14] provide the canonical survey of FL open problems and design space, including privacy-preserving variants. We treat their taxonomy as the reference for deployment-grade FL terminology.

3.2 Differential privacy in deep learning

Foundations. Differential privacy was introduced by Dwork [15] as a quantitative bound on what an adversary holding the output of an algorithm can learn about any single input record. The Gaussian mechanism is the canonical noise-addition primitive used here.

DP-SGD. Abadi et al. [2] adapted DP to deep-learning training by clipping per-sample gradients before adding calibrated Gaussian noise. The framework uses this exact construction at the local-training step (§2.2.1). The two follow-ups that drive accountant tightness are Mironov's Rényi differential privacy [3] (which gives the tightest known (ε, δ) bounds for Gaussian DP-SGD) and the Privacy Random Variable accountant [16] (slightly tighter still, supported by Opacus as an alternative); the framework uses RDP by default.

DP-FedAvg. McMahan et al. [17] extended DP-SGD to the federated setting with user-level privacy guarantees, applying noise at the server-side aggregation step rather than the client-side gradient step. Our construction places noise at the client side (closer to standard DP-SGD), which gives *record-level* DP within each client; user-level DP would require either client-side sampling at session boundaries or server-side noise. The trade-off is discussed in Chapter 9, §7.4.

Opacus. The Yousefpour et al. Opacus library [5] is the framework's DP-SGD implementation. Opacus contributes the per-sample-gradient hook machinery, the RDP accountant, and the API for auto-calibrating σ to a target ε. The framework's `differential_privacy.attach_dp` is a thin wrapper around `opacus.PrivacyEngine.make_private_with_epsilon`.

Wei et al. on DP-FL. Wei et al. [18] provide the most thorough empirical study of DP-FL utility-privacy tradeoffs to date. They report that DP-FL on balanced benchmarks pays a small, predictable utility cost; our work extends their analysis to imbalanced healthcare data and finds a structurally different cost (class-prior collapse, Chapter 8).

3.3 Secure aggregation

Bonawitz et al. pairwise mask. The secure-aggregation protocol of Bonawitz et al. [4] is the framework's direct ancestor. The protocol uses Diffie–Hellman key exchange to derive pairwise seeds, expands the seeds via a PRG into mask vectors, and uses Shamir secret-sharing of the seeds to tolerate up to t client dropouts per round. Each client masks its update; the masks cancel under summation, so the server sees only the aggregate. Our prototype implements the cancellation property (§2.2.2) but not the dropout tolerance — the simulation never drops a client.

Subsequent SecAgg variants. A line of follow-up work optimises SecAgg in different directions: SecAgg+ [19] reduces communication via a polylogarithmic communication graph; LightSecAgg [20] eliminates the quadratic pairwise-mask cost via single-mask broadcast and Lagrange-coded computing. These are deployment-grade upgrades that the prototype does not need in order to demonstrate the cancellation property.

TEE-based alternatives. Trusted-execution-environment-based schemes (Intel SGX, AMD SEV) provide aggregation confidentiality without cryptographic masking, at the cost of a hardware trust assumption. We do not use them because the framework is hardware-agnostic by design.

3.4 Gradient compression

Communication efficiency in FL. Konečný et al. [11] introduced the structured-update and sketched-update families that compress FL traffic by two orders of magnitude with bounded utility loss. Our top-k+int8 scheme is the unstructured-sparsification + scalar-quantisation member of that family.

signSGD and 1-bit SGD. Bernstein et al. [21] push compression to its extreme: transmit only the sign of each gradient component. signSGD trades bandwidth for variance and has known convergence issues in non-IID FL, which is why we use top-k instead.

Top-k sparsification. Aji and Heafield [22] demonstrated that retaining only the $\sim$1% largest-magnitude gradient components per layer preserves training quality on language modelling. We use $k = 0.1$ (10%) which is conservative for an imbalanced classification workload where rare-class gradients are already small.

Compression with privacy. Top-k sparsification interacts with DP-SGD in subtle ways: the noise that DP adds at the gradient step becomes concentrated in a smaller index set when post-hoc sparsified, amplifying its effect on the surviving entries. This is consistent with our Finding 6.4 (§8.5): top-k + int8 on top of DP-SGD pushes the model back across the cliff on records 200–234, where DP-SGD alone clears it.

3.5 Privacy-preserving FL in healthcare

FL on ECG. Kachuee et al. [9] provide the centralized baseline we reproduce; their reported test accuracy on MIT-BIH is in the 0.97–0.99 range, consistent with the framework's 0.988 (§6.3). The published FL-on-ECG papers we surveyed report similar headline accuracy under FedAvg or its variants but do *not* report majority-class baseline alongside top-1 accuracy — the editorial gap that motivates Finding 6.1 of this thesis.

FL on healthcare more broadly. Rieke et al. [23] survey the deployment landscape of federated learning across medical imaging, EHR, and wearable health data, and identify class imbalance as one of three open challenges (the other two being client heterogeneity and protocol robustness). Our class-prior collapse finding is a concrete instance of the imbalance challenge they flag.

Regulatory framing. HIPAA (Health Insurance Portability and Accountability Act 1996, US Public Law 104-191) and GDPR (Regulation (EU) 2016/679) are the two regulatory regimes that motivate the thesis's data-minimisation framing. Federated learning is a natural technical instantiation of the data-minimisation principle: training data does not leave the device.

3.6 Attacks: membership inference and gradient inversion

Membership inference. Shokri et al. [13] demonstrated that an adversary with black-box query access to a trained model can determine, with non-trivial accuracy, whether a given record was in the training set. This is the canonical attack against adversary A4 in Chapter 4, and the attack DP is built to defeat. The Yeom et al. [24] privacy-loss formalisation maps membership-inference advantage to a function of ε, providing a quantitative connection between the DP budget and attack success.

Gradient inversion. Geiping et al. [25] showed that an honest-but-curious server can reconstruct individual training images with high fidelity from the per-client gradient updates of a federated round, even under multi-step FedAvg-style local training. The attack generalises beyond images to other modalities and is the canonical attack against adversary A1 in Chapter 4. SecAgg is the canonical defence: under SecAgg the server only ever sees the aggregate $\sum_i \Delta w_i$, and gradient inversion against the aggregate is a strictly harder problem than against a single client's update.

Property inference. Beyond per-record attacks, property inference [26] extracts aggregate statistics about the training set (e.g. the gender ratio of contributing patients) from model updates. The DP and SecAgg layers both partially mitigate property inference; a complete defence would require formalising group-level DP, which is future work.

3.7 What is not covered

The framework does not engage with three adjacent literatures that the threat model deliberately excludes (Chapter 4): Byzantine-robust aggregation [27, 28] (median, trimmed mean, Krum), free-rider detection (clients that submit zero or replay updates), and verifiable computation (cryptographic proof that the server aggregated correctly). All three are listed in §9.5 as natural extensions that compose cleanly on top of the existing privacy stack.

Chapter 4

Threat model

The privacy stack of this framework is only as meaningful as the adversary it is built against. This chapter enumerates the assets being protected, the adversaries the framework defends against, the capabilities each is granted, which component of the privacy stack mitigates each threat, and the trust assumptions inherited from the deployment substrate. The threat model is the contract that the design choices in Chapter 5 are built to honour, and the limitations called out in Chapter 9 are the places where the contract is honoured only partially.

4.1 Assets being protected

Five categories of asset flow through the framework, each with its own sensitivity profile (Table 4.1). Raw waveforms and labels are both protected health information (PHI) under HIPAA and personal data under GDPR; per-client gradient updates leak training data via gradient-inversion attacks even though they are nominally derived data; the aggregated global model is the published artefact that membership-inference attacks operate on; patient and device identifiers are quasi-identifiers that must not be re-linkable to the local partitions.

Table 4.1: Assets the framework protects and their sensitivity. The privacy stack is responsible for preventing leakage of every row except patient identity, which is handled at the deployment layer (no identifier ever leaves the device in our protocol).

Asset	Sensitivity
Raw ECG waveforms	PHI under HIPAA; highly sensitive
Local labels (arrhythmia class)	Diagnostic; highly sensitive
Per-client model updates (Δw_i)	Leaks training data via gradient inversion
Aggregated global model w_t	Leaks via membership inference
Patient / device identity	Quasi-identifier; must not be re-linkable

4.2 Adversaries and capabilities

Five adversary archetypes are considered. The first four are inside the contract; the fifth is explicitly out of scope and motivates a future-work item in Chapter 9.

4.2.1 A1 — Honest-but-curious aggregation server

Capability. Sees every message that reaches the server: per-client parameter updates, per-client metadata (number of training samples, training loss, latency telemetry), broadcast parameters, and round-level scheduling information.

Behaviour. Follows the protocol faithfully. Does not deviate from the aggregation rule, drop messages, or inject crafted messages.

Goal. Infer training-data properties of any individual client from its update — e.g. reconstruct individual training records from Δw_i via gradient-inversion attacks, or learn whether a particular patient's beats appear in client i's partition.

Mitigation. Pairwise-mask secure aggregation (Chapter 5, §5.7). Under SecAgg, the server's view of every per-round message is a uniformly random vector; only the sum across all clients de-randomises into the true aggregate of deltas. The server therefore has access to $\sum_i \Delta w_i$ but not to any individual Δw_i.

4.2.2 A2 — Curious-client coalition (size $t < N$)

Capability. A subset of clients colludes among themselves. They pool their private masks (or shared seeds in the prototype), their broadcast parameters, and the metadata they each receive from the server.

Behaviour. Trains honestly so as not to be detected by simple correctness checks (the framework's SecAgg is not currently correctness-verifiable from the server's side; that is future work).

Goal. Unmask a non-coalition client's update.

Mitigation. Pairwise masks between every unordered client pair $\{i, j\}$ mean that a coalition of size t learns nothing about non-coalition updates as long as at least one mask between an honest pair survives the collusion. With $N = 4$ and pairwise masks the basic scheme withstands a coalition of up to $t = N - 2 = 2$ without compromise; with $t = N - 1 = 3$ the scheme leaks the remaining update because the only mask the honest client uses is one whose seed is held by a coalition member.

4.2.3 A3 — Network eavesdropper

Capability. Passive observation of the wire between client and server (e.g. an on-path attacker on Wi-Fi, an ISP, or a nation-state adversary at the network layer).

Goal. Same as A1, but with the additional restriction that the adversary does not control the server's storage or compute.

Mitigation. Transport-layer encryption (TLS, assumed; out of scope for the prototype but standard in any production deployment). The masked SecAgg payload is also a uniformly random vector to a network observer, providing defence-in-depth: even if TLS is subsequently broken, A3's view is no better than A1's, which SecAgg already addresses.

4.2.4 A4 — External adversary holding the released global model

Capability. Inspects, queries, or fine-tunes the released global model weights w_T. Does not see per-round messages. Has access to auxiliary information about all but one of the training records (the worst-case threat model for differential privacy).

Goal. Membership inference: "was patient P's data in the training set?" or its quantitative variant, "what fraction of patients in this hospital appear in the training set?". Reconstruction: recover features of the training distribution beyond what is published.

Mitigation. **Differential privacy** with budget (ε, δ) at the per-record level. With $\varepsilon = 5$ and $\delta = 10^{-5}$ (the framework's default targets) the worst-case posterior shift on any single-record membership question is at most $e^{\varepsilon} \approx 148$, and empirical membership-inference accuracy is typically much closer to the 50% random-guess baseline. The cost of this guarantee on imbalanced healthcare data is the central finding of Chapter 8.

4.2.5 A5 — Malicious (Byzantine) client

Capability. Deviates arbitrarily from the protocol: submits crafted updates designed to poison the global model, install a backdoor, or break SecAgg by sending mismatched mask contributions.

Goal. Either degrade the global model's utility, install a trigger pattern that produces attacker-chosen outputs at inference time, or extract information from honest clients by exploiting protocol deviations.

Out of scope. The framework's threat model does not include A5. Robust aggregation rules (Krum, coordinate-wise median, trimmed mean, multi-Krum) and protocol-level integrity checks would be the natural defences; both are listed as future work in Chapter 9, §9.5. The simulation never produces an A5: the four clients are honest by construction.

4.3 Attack-surface coverage matrix

Table 4.2 maps the five canonical attacks against federated learning to which adversary mounts them and which layer of the privacy stack mitigates each one. The matrix is the most compact statement of the contract: every cell either references a mitigation or is explicitly out of scope. There are no implicit guarantees.

Table 4.2: Attack-surface coverage. Cells marked *SecAgg* are mitigated by the pairwise-mask layer, cells marked *DP* by differential privacy, cells marked *TLS+SecAgg* by both, and cells marked *OOS* are out of scope (Byzantine adversary A5).

Attack	A1	A2	A3	A4	A5
Gradient inversion	SecAgg	SecAgg	TLS+SecAgg	—	OOS
Per-update membership inference	SecAgg	SecAgg	TLS+SecAgg	—	OOS
Global-model membership inference	DP	DP	—	DP	OOS
Reconstruction from compressed update	SecAgg	SecAgg	TLS+SecAgg	—	OOS
Model poisoning / backdoor	—	—	—	—	OOS

Why compression does not weaken the matrix. Top-k + int8 compression sits between local DP-SGD and SecAgg masking on the client (Chapter 5, §5.7). Because SecAgg is applied *after* compression, the wire payload is still a uniformly random vector to A1/A2/A3, and the compressed update is never visible in the clear. DP's post-processing immunity ensures that compression does not weaken A4's bound on the released global model. Compression is a bandwidth optimisation, not a privacy defence; it is included in the matrix only to make explicit that it does not subtract from the matrix either.

4.4 Trust assumptions

The framework's guarantees are conditional on four trust assumptions inherited from the deployment substrate:

1. **TLS is in place between every client and the server** — standard mTLS in any production deployment. The simulation does not exercise this; in a real IoT deployment it is non-negotiable.

2. **The pseudo-random generator used to derive masks is cryptographically secure.** The prototype uses Python's `secrets` module for seed generation and a deterministic NumPy generator from that seed for mask material; the latter is sufficient for simulation but a production deployment would seed an authenticated stream cipher (e.g. AES-CTR) instead.

3. **No more than $N-2$ clients collude.** The basic pairwise-mask scheme has no Shamir secret-sharing fallback (a deliberate prototype simplification, called out in `src/privacy/secure_agg.py`), so a coalition of $N-1$ clients reconstructs the remaining client's update.

4. **Hardware-level side channels (timing, power, EM) on the IoT device are out of scope.** A real wearable deployment would also need hardware countermeasures; without them, an attacker with physical access to a device can extract the local model or training data regardless of what the FL protocol does.

4.5 What is explicitly out of scope

Beyond A5, four classes of guarantee are explicitly not provided:

- **Byzantine robustness / poisoning defences.** Future work (§9.5); the natural extension is robust aggregation (Krum, median, trimmed mean) composed with the existing privacy stack.

- **Hardware-level side channels.** Out of scope at the protocol layer; addressed at the deployment layer with hardware countermeasures.

- **Verifiable computation** (cryptographic proof that the server aggregated correctly). The framework trusts that the server runs the strategy code it claims to run; deployments in adversarial settings would add zk-SNARK or trusted-execution-environment-based attestation.

- **Free-rider detection.** A client that submits zero-update vectors round after round contributes nothing to the global model but receives the benefit of others' updates; the framework does not detect or penalise this. Reputation and contribution-tracking systems are the standard defence; we do not implement either.

These are flagged again in the conclusions chapter as natural extensions that fit cleanly on top of the existing privacy stack.

Chapter 5

Method

This chapter describes the implementation that produces every number in the thesis. All code lives under `src/` and `experiments/`; YAML configs in `configs/` are the canonical record of every run. The repository is at https://github.com/mapeke/privacy-preserving-fl-healthcare-iot.

5.1 Unified client interface (`src/client/flower_client.py`)

`FlowerHealthcareClient` subclasses `flwr.client.NumPyClient` with three methods. `fit` performs local training (optionally DP-SGD), computes the parameter delta against the broadcast parameters, optionally compresses it, optionally adds the SecAgg pairwise masks, and returns reconstructed parameters (broadcast + processed delta) so vanilla Flower strategies remain compatible. `evaluate` reports per-client test loss and accuracy. `get_parameters` serialises the model state into the wire format. The class takes a single `ClientConfig` dataclass with mutually-orthogonal fields `dp_config`, `secagg_session`, `compression_method`, each of which is `None` when the corresponding defense is off.

5.2 Data layer (`src/data/`)

`mitbih_loader.py` downloads MIT-BIH records via `wfdb.dl_database` into `data/cache/`, segments each record into 360-sample beat windows centered on R-peak annotations, applies per-record z-score normalisation, and maps annotation symbols to AAMI labels. A synthetic-ECG fallback exists for offline development; both real-data and synthetic-data runs go through the same downstream pipeline.

`partitioner.py` distributes beats across N clients via Dirichlet sampling per class with concentration α (default 0.5; lower $\Rightarrow$ more skewed). For our 4-client setup the realised partition sizes are $(1907, 8621, 6947, 2840)$ — non-IID by construction.

`preprocessing.py` provides a Butterworth bandpass filter (0.5–40 Hz) for baseline-wander and powerline rejection, a stratified train/test split, and the `TorchEcgDataset` adapter.

5.3 Models (`src/models/`)

`EcgCNN` is a 3-block 1-D CNN with GroupNorm (used in place of BatchNorm for Opacus compatibility), 64 final channels, adaptive average pooling, and a 32-unit MLP head; ∼50k parameters. `LightweightEcgCNN` is a depthwise-separable variant at ∼5k parameters for resource-constrained edge inference. All experiments in this thesis use `EcgCNN`.

ReLU non-inplace. `nn.ReLU(inplace=True)` is incompatible with Opacus's per-sample gradient hooks (the in-place op writes through a backward-hook view). The exception is

swallowed by the Ray worker process in Flower's simulation backend, producing silently-flat training. Switching to non-inplace ReLU fixed this; see commit `40dd3f4`.

5.4 Differential privacy (`src/privacy/differential_privacy.py`)

`DPConfig` is the user-facing dataclass with `target_epsilon`, `target_delta`, `max_grad_norm` (the per-sample clip threshold C), optional `noise_multiplier`, and `accountant` ($\in$ {`rdp`, `gdp`, `prv`}). `attach_dp(model, optimizer, dataloader, config, epochs)` wraps the optimizer and dataloader via `opacus.PrivacyEngine`. When `noise_multiplier` is left unset Opacus auto-calibrates σ to hit `target_epsilon` over the requested epochs; otherwise the user-supplied σ is used. The returned handle exposes `get_epsilon(steps)` for per-round budget reporting.

Algorithm 1 renders the local DP-SGD step in pseudocode. The clip-then-add-Gaussian-then-average pattern is the construction §2.2.1 formalises; Opacus's `PrivacyEngine` implements lines 2–5 transparently after `attach_dp` has wrapped the model, optimiser, and dataloader.

Algorithm 1 Local DP-SGD step on minibatch B

Require: model weights w, learning rate η, clip C, noise multiplier σ

1: $g_i \leftarrow \nabla_w \ell(w; x_i, y_i)$ ▷ per-sample gradients, $i \in B$
2: $\bar{g}_i \leftarrow g_i \,/\, \max(1,\, \|g_i\|_2/C)$ ▷ ℓ_2 clip
3: $\hat{g} \leftarrow \sum_{i \in B} \bar{g}_i + \mathcal{N}(0,\, \sigma^2 C^2 I)$ ▷ noise addition
4: $\tilde{g} \leftarrow \hat{g} \,/\, |B|$ ▷ average over batch
5: $w \leftarrow w - \eta\, \tilde{g}$ ▷ SGD step
6: **return** w

5.5 Secure aggregation (`src/privacy/secure_agg.py`)

`setup_session(num_clients, seed_size_bytes=32, rng_seed=...)` generates one fresh secret seed per unordered client pair, stored in a `SecAggSession` object distributed by the experiment orchestrator. `derive_mask(seed, shape)` expands a pair seed into a Gaussian mask vector via NumPy's PCG64. `mask_update(update, client_id, session)` adds $+m_{ij}$ for $j > i$ and $-m_{ij}$ for $j < i$ to the client's update, so that summing all clients' masked updates cancels every mask exactly. `aggregate_masked_updates` performs the server-side sum on the strategy side (§5.7).

Algorithm 2 renders the per-round client-side masking and the server-side aggregation. The cancellation property (Equation 2.2, §2.2.2) is encoded in lines 4–5: the asymmetry between $\sum_{j>i}$ and $\sum_{j<i}$ is what makes every mask appear once with $+$ and once with $-$ across the full set of clients.

Limitations explicitly accepted. Pre-shared seeds rather than Diffie–Hellman; no Shamir secret-sharing (no dropout tolerance); float32 arithmetic rather than modular integer over a finite field. These are appropriate for a diploma simulation that wants to demonstrate the principle and measure end-to-end overhead, and they are flagged in the module docstring.

5.6 Compression (`src/privacy/compression.py`)

`compress(update, method, top_k_ratio, quant_bits)` dispatches over four modes: `none`, `top_k_only`, `quantize_only`, `top_k_quantize`. `top_k_sparsify` keeps the largest-magnitude $\lceil k|u| \rceil$ entries via $O(n)$ argpartition. `quantize_int8` performs min–max quantisation with a documented `(scale, zero_point)` pair; `decompress` reconstructs a dense update array (zeros

Algorithm 2 Pairwise-mask SecAgg: client i masks, server aggregates

Require: client update Δw_i, session $\{s_{ij}\}_{j \neq i}$, total clients N

```
function MaskUpdate(Δw_i, i, session)
    m ← 0_d
    for j ∈ {1, ..., N} \ {i} do
        if j > i then    m ← m + derive_mask(s_ij, d)
        else   m ← m − derive_mask(s_ji, d)
        end if
    end for
    return Δ̃w_i ← Δw_i + m
end function

function ServerAggregate({Δ̃w_i}_{i=1}^N)
    S ← Σ_{i=1}^N Δ̃w_i                          ▷ masks cancel by construction
    return S / N                                ▷ average; the wire weight n_i folds in here
end function
```

where sparsified). The wire payload is a `CompressedUpdate` dataclass with `indices`, `values`, `original_shape`, and a `num_bytes()` method for bandwidth accounting.

Algorithm 3 renders the round-trip. The `(scale, zero_point)` quantisation parameters travel alongside the `values` on the wire; without them the receiver cannot dequantise back into floating point, so the wire payload is indices $\times$ values $\times$ (scale, zero_point). Bandwidth-wise, this dominates over float32 by $\sim$40$\times$ at $k = 0.1$ even after accounting for the index overhead.

Algorithm 3 Top-k + int8 compression round-trip

Require: update vector $u \in \mathbb{R}^d$, sparsity $k \in (0, 1]$

```
function Compress(u, k)
    K ← ⌈k d⌉
    I ← argpartition(|u|, d − K)[d − K:d]        ▷ top-K indices, O(d)
    V_f ← u[I]                                   ▷ floating-point values at those indices
    scale ← (max V_f − min V_f) / 255
    zp ← round(− min V_f / scale)
    V_q ← clip(round(V_f/scale) + zp, 0, 255)    ▷ int8
    return (I, V_q, scale, zp, d)
end function

function Decompress(I, V_q, scale, zp, d)
    û ← 0_d
    û[I] ← (V_q − zp) · scale
    return û                                     ▷ û ≈ u; zeros at Ī
end function
```

5.7 Server strategy (`src/server/strategies.py`)

`PrivacyAwareFedAvg` subclasses Flower's `FedAvg`. With SecAgg disabled it falls through to standard weighted averaging. With SecAgg enabled, `configure_fit` caches the broadcast parameters; `aggregate_fit` recovers per-client deltas by subtracting them from each client's returned parameters, sums the deltas (where masks cancel by construction), divides by the number of participating clients, and applies the result on top of the cached broadcast. Robust

scalar-metric aggregation (sample-weighted mean for numerics, first-occurrence for non-numerics) covers the case where Flower's default aggregator chokes on heterogeneous metric dictionaries.

5.8 Experiment infrastructure

Five entry points, each a thin CLI wrapper around `experiments/_common.py::build_experiment` + `run_federated_experiment`:

- `run_centralized.py` — pooled-data baseline.
- `run_federated.py` — vanilla FedAvg.
- `run_federated_dp.py` — FedAvg + DP-SGD.
- `run_federated_secagg.py` — FedAvg + SecAgg.
- `run_full_pipeline.py` — DP + SecAgg + compression.

Each writes `results/<name>/{run.log, resolved_config.yaml, history.json}` so every reported number can be traced back to its config.

5.9 Tests

42 pytest unit tests across seven modules; the most consequential is `test_secagg.py::test_masks_cancel_unde` which constructs four random updates, masks them, sums the masked vectors, and asserts that the result equals the unmasked sum to within 10^{-4} in ℓ_2. This is contribution C2: the cryptographic correctness argument as a code-level invariant.

The test suite passes in 4.0s on the developer machine. `Opacus` is imported lazily inside `attach_dp` so the tests run without it installed; the test that asserts that lazy-import property is itself an invariant.

5.10 Per-round data flow

The previous sections describe each component in isolation. This section walks through what happens in one federated round end-to-end — the path that every byte of every update traverses — so the composition of DP-SGD, compression, masking, aggregation, decompression, and update travels visibly across the per-component sections instead of being implicit.

A round is the unit of progress in Flower's `start_simulation` loop. Each round consists of nine numbered steps; the configuration knobs that gate each step are listed alongside.

1. **Server samples clients.** `configure_fit` on `PrivacyAwareFedAvg` (§5.7) selects a fraction `fraction_fit` of the N available clients and caches the broadcast parameters w_t for the SecAgg unmasking path.
2. **Server broadcasts w_t.** Flower serialises w_t to NumPy arrays and dispatches to each sampled client via `client_fn(cid)`.
3. **Client local training.** The client constructs an `EcgCNN` and a `DataLoader` from its private partition. If `dp_config` is set, `attach_dp` (§5.4) wraps the optimiser and dataloader with Opacus's `PrivacyEngine`. `local_epochs` passes are then run; each minibatch step is Algorithm 1 (with DP) or vanilla SGD (without).
4. **Client computes the delta.** $\Delta w_i \leftarrow w_i - w_t$ is the update payload; the cached broadcast parameters w_t are kept by the strategy so the server can later reconstruct $w_{t+1} = w_t + \overline{\Delta w}$.

5. **Client compresses Δw_i (optional).** If `compression_method` is set, `compress` (§5.6, Algorithm 3) returns $(I, V_q, \text{scale}, \text{zp}, d)$. The compression is applied per-tensor, not on the flattened vector, so the index payload scales with the per-layer $K = \lceil k\, d_\ell \rceil$.

6. **Client masks Δw_i (optional).** If `secagg_session` is set, `mask_update` (§5.5, Algorithm 2) is applied to the post-compression vector. The masked-and-compressed payload is the wire message.

7. **Server aggregates.** `aggregate_fit` on `PrivacyAwareFedAvg` reconstructs each client's $\tilde{\Delta w}_i$ by subtracting w_t from the returned parameters. With SecAgg enabled it sums the $\tilde{\Delta w}_i$ (masks cancel by Algorithm 2); without SecAgg it falls through to Flower's vanilla weighted average. The output is $\overline{\Delta w} = \frac{1}{N} \sum_i \Delta w_i$.

8. **Server decompresses (optional).** If compression was applied, the per-client deltas are decompressed via `decompress` (Algorithm 3) before averaging. The framework currently averages decompressed dense vectors rather than the more efficient decompress-after-summing-sparse, because the SecAgg layer requires dense vectors to mask in the first place.

9. **Server updates the global model.** $w_{t+1} \leftarrow w_t + \overline{\Delta w}$. The new w_{t+1} is cached, evaluation is dispatched if the round is an evaluation round, and the loop returns to step 1 for round $t + 1$.

Where the privacy layers attach. Steps 3, 5, and 6 are the three privacy switches (`dp_config`, `compression_method`, `secagg_session`). The framework allows each to be `None`, in which case the corresponding step degenerates to the identity — this is what makes the per-layer cost measurements in Chapters 6–8 possible without code branches.

Where the threat model attaches. The on-wire payload between steps 6 and 7 is the SecAgg-masked, compressed delta; this is what adversaries A1, A2, and A3 (§4) see. The released global model w_T at the end of T rounds is what adversary A4 sees. The local data on each client never crosses the wire; this is the data-minimisation property that motivates the framework (Chapter 1, §1.1).

Chapter 6

Results I — Centralized and federated baselines

Setup: 12 MIT-BIH records (Table 2.2), 4 clients, Dirichlet $\alpha = 0.5$ partitions of sizes $(1907, 8621, 6947, 2840)$, 80/20 stratified train/test split. Configs: `configs/default.yaml`. Hardware: single NVIDIA RTX (mobile) GPU, CUDA 12.1, PyTorch 2.4.1. Seed 42.

6.1 Detection grid

Table 6.1 reports the headline accuracy for every configuration we run in this thesis. The remainder of this chapter establishes the top two rows; Chapters 7 and 8 establish the rest.

Table 6.1: Final test accuracy across all five experimental configurations. Federation is free; SecAgg is free; DP collapses to the majority-class prevalence; compression on top of DP does not move the needle further. Numbers are single-seed at seed 42 to match the thesis policy of "every number tied to one config row."

Configuration	Test acc	Macro-F1	Final ε
Centralized (5 epochs)	**0.9878**	0.36	—
FedAvg, 4 clients, 10 rounds	**0.9880**	—	—
FedAvg + SecAgg, 10 rounds	0.9860	—	—
FedAvg + DP, 20 rounds, target $\varepsilon = 5$	0.9711	—	4.9955
Full pipeline (DP+SecAgg+Comp), 30 rounds, target $\varepsilon = 8$	0.9711	—	7.9964
Class-N prevalence (= predict-all-N)	0.9716	0.20	—

6.2 Centralized baseline

5 epochs of plain SGD on the pooled training set (20,315 beats), $\eta = 0.01$, momentum 0.9, weight decay 10^{-4}, batch size 32. Final test accuracy 0.9878, macro-F1 0.3627. Per-class F1: $\{N{=}0.99,\ V{=}0.82,\ S{=}0.00,\ F{=}0.00,\ Q{=}0.00\}$. The model learns the two prevalent classes (N, V) and ignores the others. Macro-F1 is $\sim$0.36 because three of the five classes contribute zero, a structural property of the workload.

6.3 Vanilla FedAvg

10 rounds, 4 clients per round, 2 local epochs, batch 32, $\eta = 0.01$. Table 6.2 gives the round-by-round test accuracy.

Table 6.2: Vanilla FedAvg test accuracy round by round on records 100–115 (`results/vanilla_fedavg/history.json`). The first two rounds match class-N prevalence (0.9711) before the model picks up the V-class signal in round 3; the small dip at round 6 is within seed variance.

Round	1	2	3	4	5	6	7	8	9	10
Acc.	0.9711	0.9711	0.9758	0.9840	0.9840	0.9724	0.9831	0.9864	0.9862	**0.9880**

Finding 4.1 — Federation is free on this workload. Final FedAvg accuracy 0.9880 vs. centralized 0.9878. The federated decomposition introduces zero accuracy cost up to seed variance. No special tuning, no proximal regularizer [10], no server-momentum — plain FedAvg suffices when client data is similar enough in distribution and the model is not over-parameterised.

This is the empirical baseline against which Chapters 7 and 8 measure the cost of the privacy stack. Any utility loss observed with SecAgg or DP enabled is attributable to the privacy mechanism, not to the federation itself.

6.4 Reproducibility check vs. published FL ECG results

Single-seed run; we did not run multi-seed for the headline figure due to GPU-time budget (each FedAvg run is ∼3 minutes on our hardware, but DP and full-pipeline runs are 30 minutes each; multi-seed across all five configurations is a future-work item, §9.5). Published FL ECG papers report top-1 accuracy in the 0.97–0.99 range on similar MIT-BIH subsets [9]; we are within that range. The thesis's contribution does not depend on matching any published absolute — it depends on the relative shape across the privacy stack rows.

Chapter 7

Results II — Secure aggregation

Configs: `configs/experiment_secagg.yaml`. SecAgg enabled, DP off, compression off. 10 rounds. Otherwise identical to Chapter 6.

7.1 Mask cancellation as a code-level invariant

Finding 5.1 — Cryptographic correctness is mechanically verifiable. The pairwise-mask correctness argument [4] is reduced to one assertion:

> *for* $N = 4$ *random updates of shape* $(10,)$, $\sum_{i=1}^{N} \text{mask_update}(u_i, i, \text{session}) = \sum_{i=1}^{N} u_i$ *to within* 10^{-4} ℓ_2 *error.*

This lives in `tests/test_secagg.py::test_masks_cancel_under_aggregation` and runs in CI alongside the rest of the pytest suite. The same test verifies the dual property: each individual masked update is *not* close to the unmasked update (the masks are non-trivial; the cancellation only happens at the sum level).

The contribution here is not the test itself — it is one screenful of NumPy — but the discipline of writing the protocol's correctness argument as a checked invariant rather than as prose. A future change that breaks the protocol (e.g., an off-by-one in the $i < j$ vs. $i > j$ sign condition) fails the test loudly rather than silently producing biased aggregates.

7.2 End-to-end SecAgg run

Round-by-round test accuracy is in Table 7.1; final 0.9860.

Table 7.1: FedAvg + SecAgg test accuracy round by round on records 100–115 (`results/federated_secagg/history.json`), placed alongside vanilla FedAvg from Table 6.2 for direct comparison. SecAgg lags vanilla FedAvg by a small, noise-floor-consistent margin throughout.

Round	1	2	3	4	5	6	7	8	9	10
FedAvg	0.9711	0.9711	0.9758	0.9840	0.9840	0.9724	0.9831	0.9864	0.9862	0.9880
+ SecAgg	0.9711	0.9711	0.9711	0.9776	0.9837	0.9840	0.9844	0.9848	0.9846	0.9860

Per-class F1 not logged round by round. The framework's evaluation hook only writes top-1 accuracy per round to `history.json`; macro-F1 and per-class precision/recall are computed at the final round only. Round-by-round F1 logging is on the future-work list (§9.5); the diagnostic value would have been to spot the rare-class signal (or lack thereof) earlier than the final-round

metric reveals it. We do not retroactively recompute F1 from the saved checkpoints because that would require re-running evaluation on the held-out set and would not contribute material absent from the existing single-seed run.

Finding 5.2 — SecAgg overhead is below run-to-run noise. Vanilla FedAvg final 0.9880 vs. FedAvg + SecAgg final 0.9860 — a 0.20 pp gap. Per-class F1, train accuracy, and convergence trajectory all overlap with vanilla FedAvg's within a few hundredths. The 0.20 pp gap is consistent with float32 accumulation noise in the masking; it would shrink under integer modular arithmetic (future work, §9.5).

The combination of §7.1 and this finding is the case for SecAgg as a default in any FL deployment: the cryptographic correctness is a one-line CI check; the utility cost is below the noise floor; and the threat model that SecAgg addresses (honest-but-curious server with read access to the inbox) is realistic for any cloud-hosted aggregation server.

7.3 What SecAgg does not protect against

SecAgg blinds the server to per-client updates but does not blind it to the *aggregate*. An adversary holding the released global model w_T can still mount membership-inference attacks against training data; that threat is what differential privacy (Chapter 8) addresses. The two layers are complementary, and the thesis argues for running both, modulo the utility cost which Chapter 8 now measures.

Chapter 8

Results III — Differential privacy and the full pipeline

Configs: `configs/experiment_dp.yaml` and `configs/experiment_full.yaml`. DP enabled (Opacus PrivacyEngine, RDP accountant, `max_grad_norm = 1.0`). Otherwise identical to Chapter 6.

8.1 DP-SGD run

20 rounds, 1 local epoch per round, batch size 64, $\eta = 0.005$, target $\varepsilon = 5.0$, target $\delta = 10^{-5}$. Opacus auto-calibrates σ to hit $\varepsilon = 4.9955$ (well within the target).

Round-by-round test accuracy is exactly flat to four decimal places across all 20 rounds: every round reports 0.9711, identical to the four significant figures of class-N prevalence (0.9716; §8.2). Table 8.1 gives the trajectory.

Table 8.1: DP-SGD ($\varepsilon \approx 5$) test accuracy round by round on records 100–115 (`results/federated_dp/history.json`). The model is the majority-class predictor from round 1 onward; train accuracy rises 0.9147 → 0.9712 across the same 20 rounds, but evaluation accuracy never moves.

Round	1	2	3	4	5	6	7	8	9	10
Acc.	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711
Round	11	12	13	14	15	16	17	18	19	20
Acc.	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	0.9711	**0.9711**

Train metrics tell the inner story: train loss falls 0.5370 → 0.2233; train accuracy rises 0.9147 → 0.9712. The model is doing *something* during training; that something just happens to be "learn to predict N and only N."

8.2 Class-prior collapse

Finding 6.1 — DP-SGD collapses to majority-class prediction. Class N prevalence in the test split: 4,934/5,078 = 0.9716. Final DP-SGD test accuracy: 0.9711. The two numbers agree to four decimals. The model is, with statistical certainty, the majority-class predictor.

This is not a learning-rate or noise-multiplier failure — the train accuracy converges to the same 0.9712, and the spent ε is within the target. It is a workload-mismatch failure: DP-SGD's per-step Gaussian noise drowns out the gradient signal of the rare classes, which contribute

$\sim$2.9 % of the data. Under sufficient noise, the empirical-risk-minimising policy on the training set is exactly: predict the majority. Opacus is doing what it advertises; the workload asks more of it than the budget will pay for.

Why this matters editorially. A casual reader of Table 6.1 sees "0.9711" beside "DP $\varepsilon = 5$" and concludes that DP costs the framework about 1.7 pp of accuracy — a number that would be publishable under the headline-accuracy convention dominant in the FL-privacy literature. The honest reading is that DP costs the model's ability to detect any rare class at all on this data. The 0.9711 number hides the failure precisely because the metric is overall accuracy on a heavily imbalanced label space.

The framework therefore reports macro-F1 alongside accuracy in `src/utils/metrics.py::compute_classi` the centralized run's macro-F1 of 0.36 (Chapter 6) drops to ≤ 0.20 on the DP run, which would have made the collapse visible if multi-seed F1 had been logged across rounds. We did not log it for the federated runs; we will (§9.5).

8.3 The full pipeline run

30 rounds, 1 local epoch, batch 64, $\eta = 0.005$, target $\varepsilon = 8.0$, target $\delta = 10^{-5}$, SecAgg enabled, top-k + int8 compression with $k = 0.1$. Opacus auto-calibrates σ for the looser ε, hits $\varepsilon = 7.9964$.

Round-by-round test accuracy: 0.9711 across all 30 rounds, identical to the DP-only run (Table 8.1). The model never leaves the majority-class plateau, which we read as evidence that the DP noise floor at $\varepsilon = 5$ has already removed the rare-class gradient signal — the additional approximation error from compression has nothing left to disturb.

Finding 6.2 — Compression on top of DP does not degrade further. DP-only at $\varepsilon = 5$ converges to 0.9711. Full pipeline (DP + SecAgg + compression) at $\varepsilon = 8$ converges to 0.9711. The model is already at the floor that the workload's class prior imposes; there is nothing left to degrade by adding more layers. Loosening the privacy budget from $\varepsilon = 5$ to $\varepsilon = 8$ does not move the model off the floor either, which would otherwise be the natural follow-up question.

Compositional implication. For an operator constrained to deploy DP at $\varepsilon \leq 8$ on a class-imbalanced workload, adding SecAgg and compression is essentially free: they do not make the already-collapsed model any worse. This inverts the usual "every layer costs something" framing for the specific case where the bottom layer has already maxed out the utility cost.

8.4 Negotiating the cliff

The thesis does not propose a fix; the experimental contribution is to make the cliff visible. Three directions a follow-up paper would take:

1. **Class-weighted DP-SGD.** Apply per-class loss weights so that the gradient signal from the rare classes is amplified before clipping. Risks: changes the privacy accounting because the ℓ_2 sensitivity of the loss is no longer 1.

2. **Per-class privacy budgets.** Decouple ε for majority and minority classes, e.g. $\varepsilon_N = 1$, $\varepsilon_{S,V,F,Q} = 10$. Mathematically clean but practically awkward to deploy — the privacy guarantee is then conditional on the target class, which a deployment operator may find harder to reason about than a single ε.

3. **Different MIT-BIH records.** Records 200–234 contain more arrhythmias and reduce the imbalance from 97.1 % N to 82.1 % N. We re-ran all five configurations on this slice (§8.5

below) without retuning, both to test whether the cliff is dataset-conditional and to give the rare-class classifier a fairer chance. The deployment workload (a wearable monitoring an asymptomatic patient) is closer to the 100–115 imbalance than to 200–234, so the primary slice remains the more representative one.

8.5 Follow-up: records 200–234

To test whether the DP class-prior collapse (§8.2) is a property of DP-SGD itself or of the workload's extreme imbalance, we re-ran all five experimental configurations on records 200–234 — a 25-record slice with 61,818 beats and substantially more arrhythmia variety. No hyperparameters were retuned; configs are `configs/followup_records200/{default, experiment_dp, experiment_secagg, experiment_full}.yaml`. Table 8.2 contrasts the two slices' class distributions; Table 8.3 reports final accuracies side by side with the original Table 6.1.

Table 8.2: AAMI class distribution on records 100–115 (the primary slice, 25,393 beats) and records 200–234 (the follow-up, 61,818 beats). The follow-up reduces the majority-class share from 97.1 % to 82.1 % and gives all four rare classes between 1 % and 10 % of the data each.

Class	100–115 share	200–234 share
N (normal)	97.10 %	82.08 %
S (supraventricular)	0.24 %	4.19 %
V (ventricular ectopic)	2.60 %	9.53 %
F (fusion)	0.03 %	1.28 %
Q (unknown / paced)	0.03 %	2.93 %

Table 8.3: Same five experiments, records 100–115 vs. records 200–234, same hyperparameters in both columns. The majority-class baseline (predict-all-N) is 97.16 % on 100–115 and 82.08 % on 200–234; the right-hand column is the gap from that baseline on the follow-up slice.

Configuration	100–115 acc	200–234 acc	vs. majority
Centralized (5 epochs)	0.9878	**0.9389**	+11.81 pp
FedAvg (10 rounds)	0.9880	**0.9636**	+14.28 pp
FedAvg + SecAgg (10 rounds)	0.9860	**0.9558**	+13.50 pp
FedAvg + DP, $\varepsilon \approx 5$ (20 rounds)	0.9711	**0.8845**	+6.37 pp
Full pipeline, $\varepsilon \approx 8$ (30 rounds)	0.9711	**0.8176**	−0.32 pp

Finding 6.3 — The DP class-prior collapse is dataset-conditional. On records 200–234, DP-SGD at $\varepsilon \approx 5$ converges to 0.8845 test accuracy, +6.37 pp above the 0.8208 majority-class baseline. The model has *not* collapsed: it is distinguishing rare-class beats. Compare to the original 100–115 result, where DP-SGD's 0.9711 matched class-N prevalence to four decimals. Finding 6.1 (§8.2) is therefore not a property of DP-SGD but a property of *extreme* class imbalance composed with DP's per-step noise. The cliff exists; its location is workload-dependent.

Finding 6.4 — The full pipeline collapses on moderate imbalance too. The full-pipeline run (DP + SecAgg + top-k + int8 at $\varepsilon \approx 8$) on records 200–234 converges to 0.8176, 0.32 pp *below* the 0.8208 majority-class baseline. The run does not stabilise: across the 30 rounds the test accuracy ranges from a low of 0.6062 (round 10) to a high of 0.8589 (round 6), and the final five rounds drift between 0.8302 and 0.8436 before falling back to 0.8176 at

round 30. Table 8.4 shows the trajectory in full. Top-k + int8 compression on top of DP-SGD's already-noisy gradients appears to push the regime back across the cliff that DP-SGD alone clears on this dataset.

Table 8.4: Full-pipeline (DP + SecAgg + top-k + int8) test accuracy round by round on records 200–234 (`results/followup_records200/federated_full_pipeline/history.json`). The run never stabilises; the final-round number reported in Table 8.3 (0.8176) is one point on a noisy trajectory rather than a converged value.

Round	1	2	3	4	5	6	7	8	9	10
Acc.	0.8208	0.8208	0.8208	0.8208	0.8208	0.8589	0.7093	0.7169	0.6391	0.6062
Round	11	12	13	14	15	16	17	18	19	20
Acc.	0.6478	0.7011	0.7143	0.7118	0.7456	0.7654	0.7654	0.7951	0.7973	0.7903
Round	21	22	23	24	25	26	27	28	29	30
Acc.	0.8200	0.8023	0.8287	0.8236	0.8257	0.8396	0.8436	0.8302	0.8425	**0.8176**

DP-only on records 200–234 climbs out of the majority plateau. By contrast, the DP-only run on records 200–234 starts at the majority baseline and climbs steadily across 20 rounds: Table 8.5. The model holds 0.8208 for the first three rounds (the test accuracy that predicting class N universally achieves), then breaks into the rare-class signal and plateaus around 0.88. The contrast with the records 100–115 DP run (Table 8.1, flat at 0.9711) is the empirical core of Finding 6.3.

Table 8.5: DP-only ($\varepsilon \approx 5$) test accuracy round by round on records 200–234 (`results/followup_records200/federated_dp/history.json`). Three flat rounds at the majority baseline, then a clean climb into the rare-class signal — the trajectory the records 100–115 DP run could not produce.

Round	1	2	3	4	5	6	7	8	9	10
Acc.	0.8208	0.8208	0.8208	0.8670	0.8667	0.8758	0.8691	0.8775	0.8750	0.8777
Round	11	12	13	14	15	16	17	18	19	20
Acc.	0.8770	0.8719	0.8795	0.8816	0.8775	0.8794	0.8812	0.8814	0.8813	**0.8845**

Loosening the budget from $\varepsilon = 5$ (DP-only) to $\varepsilon = 8$ (full pipeline) is not enough to compensate — the additional noise floor is the lossy compression, not the DP budget.

What the follow-up changes editorially. The Chapter 8 framing "DP collapses on imbalanced healthcare data" is true on records 100–115 but reads as a generalisation. The honest generalisation is: *DP-SGD's signal-to-noise ratio degrades sharply when the rare-class share approaches the per-step noise floor; the cliff is observable but its location depends on both the budget and the workload imbalance.* The full pipeline crosses the cliff on both slices, which is the more generalisable finding the framework supports.

What the follow-up does not change. We do not retune hyperparameters per dataset — the contribution is the relative shape across the privacy stack, not absolute accuracy on either slice. We do not claim records 200–234 is the "right" slice; the deployment workload (a wearable monitoring an asymptomatic patient) is closer to the 100–115 imbalance. Both slices are useful:

100–115 surfaces the cliff at the dataset extreme, 200–234 shows where on the class-balance axis it sits.

Chapter 9

Discussion

9.1 What the three findings add up to

- **Federation (§6.3).** Free on this workload.
- **Secure aggregation (§7.1, §7.2).** Free up to fp32 mask-cancellation noise; cryptographic correctness is a one-line CI invariant.
- **Differential privacy (§8.2).** The privacy-utility cliff is real and is the central pitfall of DP-SGD on imbalanced healthcare data. The DP run's headline accuracy looks reasonable (0.9711) precisely because the metric matches the cliff: predicting the majority class scores 0.9716.
- **Dataset dependence (§8.5).** Finding 3 is conditional on *extreme* class imbalance: on records 200–234 (82 % N rather than 97 % N) DP-SGD alone retains a +6.37 pp rare-class signal, while the full pipeline still collapses. The cliff is a property of the workload composed with the privacy stack, not of DP-SGD per se; the framework is what makes that property visible.

The thesis-level recommendation: any privacy-preserving FL paper for imbalanced healthcare data should report majority-class baseline alongside overall accuracy and macro-F1 alongside top-1; the privacy/utility tradeoff graph is misleading without them. Our framework contributes both the mechanism that produces the data and the protocol that surfaces the cliff.

9.2 Likely reviewer objections

"You only ran each configuration at one seed." True. Vanilla FedAvg, SecAgg, and DP are seed-stable on this dataset to within roughly 0.5 pp accuracy according to spot-checks during development. The DP collapse is structural rather than seed-dependent — it is the model converging to the majority-class predictor, which the same training process produces from any random initialisation. Multi-seed across all configurations is the first item of §9.5.

"You only used 12 records." The primary experiments use records 100–115 (the `wfdb.dl_database` default). §8.5 re-runs all five experiments on records 200–234 (25 records, 61,818 beats, 82.1 % N) without retuning. The qualitative finding holds with one refinement: DP-SGD alone clears the cliff on records 200–234 but the full pipeline still collapses. The 12-record default is enough to *surface* the cliff; the 25-record follow-up is what places it on the class-balance axis.

"You did not compare to FedProx or SCAFFOLD." The contribution is layering existing privacy primitives on top of FedAvg, not optimising the FL aggregation itself. FedAvg achieves 0.988 on this workload — approximately the centralized baseline. There is no headroom for a stronger FL aggregator to demonstrate on this data. A different workload (more clients, more heterogeneity) would justify the comparison.

"Your SecAgg is not Bonawitz et al." Acknowledged. The prototype uses pre-shared seeds rather than Diffie–Hellman, no Shamir secret-sharing for dropout tolerance, and float32 arithmetic. These limitations are listed in the module docstring, in `docs/privacy-analysis.md` §2, and at the start of §7. The cryptographic correctness invariant is the only contribution we claim from the SecAgg layer; that survives the simplifications.

"Why not MIMIC-IV / PhysioNet challenge data / X?" MIT-BIH is the canonical ECG benchmark; all other datasets we considered (MIMIC-IV vital signs, the PhysioNet 2020 12-lead challenge) introduce either feature heterogeneity or scale that the framework does not currently model. The framework is dataset-agnostic at the loader level (`src/data/`); switching to MIMIC-IV is a one-file extension and is listed in §9.5.

9.3 Practical implications for deployment

The framework's three findings translate into three concrete recommendations for any team operationalising privacy-preserving FL on healthcare data:

- **Run SecAgg by default.** Free utility cost (§7.2), meaningful threat coverage (honest-but-curious server, per-update tracing), and the correctness check is one unit test.
- **DP-SGD on imbalanced clinical data is non-trivial.** The headline-accuracy number can mask total class-prior collapse. Always report majority-class baseline and macro-F1.
- **Compression is free in the post-DP regime.** If the model is already at the privacy floor, top-k + int8 compression buys bandwidth at no further utility cost. If the model is *not* at the floor, compression introduces additional approximation error and must be measured explicitly.

A configuration recipe. Concretely, an operator deploying this framework on a wearable-IoT ECG classifier with ∼1000 devices and a target threat model matching A1–A4 (Chapter 4) would set the configuration in Table 9.1. The reasoning behind each value is documented in the chapter referenced; the configuration is meant as a defensible starting point, not a final tuning.

What this configuration does *not* solve. The class-prior collapse on extreme imbalance (records 100–115, 97% N) is not curable within this configuration — ε would need to be relaxed past values that are clinically meaningful, or class-weighted DP-SGD would need to be implemented (§9.5). On more moderate imbalance (records 200–234, 82% N) DP-only at the configuration above retains a +6.4 pp signal over majority but the full pipeline at $\varepsilon = 8$ does not. Operators should benchmark on their actual class distribution before fixing the budget.

9.4 What we deliberately did not test

The framework's threat model and experimental scope were chosen deliberately. Three exclusions, each with a different motivation:

Table 9.1: Suggested starting configuration for a privacy-preserving FL deployment under threat model A1–A4. Each value links back to the chapter that justifies it; the operator should re-tune `target_epsilon` per their target rare-class recall and `top_k_ratio` per their bandwidth budget.

Knob	Value	Justification
`secagg.enabled`	`true`	Below-noise utility cost; covers A1–A3 (§7.2, §4)
`dp.target_epsilon`	5.0–8.0	Conventional moderate budget; relax to 8 if rare classes matter (§2.2.1)
`dp.target_delta`	10^{-5}	$< 1/N$ for any $N \gtrsim 10^5$ (§2.2.1)
`dp.max_grad_norm` (C)	1.0	Opacus default; matches observed gradient scale (§2.2.1)
`dp.accountant`	`rdp`	Tightest known bounds for Gaussian DP-SGD (§3.2)
`compression.method`	`top_k_quantize`	Bandwidth ∼40×; free if the model is already at the DP floor (Finding 6.2)
`compression.top_k_ratio`	0.05–0.10	Conservative for imbalanced data; raise toward 0.2 if Finding 6.4 manifests
`compression.quant_bits`	8	Lossless modulo ± 1 LSB at the magnitudes we observe
`rounds`	10–30	10 if SecAgg-only, 20+ if DP enabled (Opacus needs more steps)
`local_epochs`	1–2	Lower under DP (composition cost); higher without DP
`batch_size`	32–64	64 under DP for better gradient averaging

Byzantine / poisoning attacks. The threat model (Chapter 4) is honest-but-curious server plus model-recipient adversary. Robust aggregation rules (Krum, median, trimmed mean, multi-Krum) are the canonical defences against the Byzantine adversary A5 and compose cleanly on top of the existing privacy stack: a Byzantine-robust aggregator can replace the weighted average inside `PrivacyAwareFedAvg.aggregate_fit` (§5.7) without disturbing SecAgg cancellation or DP accounting. We do not run this experiment because the simulation has no native source of malicious clients, and synthesising one would require constructing the kind of attack we are not in a position to publish without an accompanying defence evaluation ([27, 28]).

Real network-level performance. The Flower simulation is in-process. We report bandwidth in bytes-per-round (∼40× savings under top-k+int8) but never measure end-to-end latency over a real wide-area or cellular link, never simulate client-side compute pressure on the IoT device's CPU/GPU, and never exercise dropout-tolerance: every client that starts a round finishes it. A production deployment would also want power consumption per round (measured on the actual wearable's SoC), thermal throttling under sustained training, and the latency distribution of SecAgg under real-network jitter. None of these are part of the contribution; the framework is dataset- and protocol-grade, not deployment-grade.

Patient-level privacy guarantees (vs. record-level). The DP-SGD construction in this framework provides record-level DP: each ECG beat is one record, and ε bounds the influence of any single beat on the released model. In our partition each MIT-BIH record corresponds to one patient and contains thousands of beats, so the patient-level ε is $k\varepsilon$ where k is the number of beats per patient (group privacy). For our $k \approx 2000$ this is a vacuous guarantee at the patient

level. Recovering meaningful patient-level ε would require either user-level DP-FedAvg [17] (server-side noise on the aggregated update, with client-level sensitivity instead of record-level) or a sampling protocol that admits each patient only sparsely into training. Both are listed in §9.5 as the natural extension that closes the gap between the framework's record-level guarantee and the patient-level guarantee that a HIPAA/GDPR deployment audit would actually demand.

9.5 Future work

1. **Multi-seed runs across all five configurations** to put error bars on Table 6.1. Roughly 4 hours of GPU time per 3-seed sweep; limiting factor is the full-pipeline run (30 min/seed).
2. **Class-weighted DP-SGD** to test the proposed mitigation in §8.4 on the records 200–234 slice, where DP-SGD alone clears the cliff but the full pipeline does not. Records 200–234 already lifted the rare-class signal; class-weighted loss is the next dial.
3. **Modular-integer SecAgg** to remove the float32 cancellation noise observed in §7.2 and bring the SecAgg overhead to exactly zero.
4. **Byzantine-robust aggregation** (Krum, median, trimmed mean) composed with the existing privacy stack.
5. **MIMIC-IV vital-signs port** to test whether the class-prior collapse generalises to a different healthcare modality with different label distribution.
6. **Diffie–Hellman key exchange + Shamir secret-sharing** to upgrade the SecAgg prototype to a dropout-tolerant deployment-grade protocol.

Chapter 10

Conclusion

We built a layered-defense privacy-preserving federated learning framework for ECG arrhythmia classification on MIT-BIH, with three independently switchable defenses (DP-SGD, pairwise-mask SecAgg, top-k + int8 compression) on top of FedAvg. Five configurations were benchmarked on real PhysioNet data with four simulated wearable IoT clients.

Three findings define the thesis:

1. **Federation is free** on this workload — vanilla FedAvg matches centralized accuracy within 0.02 pp. The federated decomposition introduces no measurable utility cost.

2. **Secure aggregation is free** up to float32 mask-cancellation noise — 0.20 pp gap vs. vanilla FedAvg, well below run-to-run variance. The cryptographic correctness property is encoded as a unit test that runs in CI; the protocol is mechanically verified, not just argued.

3. **DP-SGD collapses to majority-class prediction** on this imbalanced workload. The final accuracy at $\varepsilon = 5$ is 0.9711, which equals the prevalence of class N (Normal) to four decimals — the model is, with statistical certainty, the majority-class predictor. Adding compression on top of DP does not degrade further because there is nothing left to degrade. A follow-up on records 200–234 (§8.5) shows that the collapse is *dataset-conditional*: on moderate imbalance (82 % N) DP-SGD alone retains a +6.37 pp signal over the majority baseline, but the full pipeline still collapses at $\varepsilon = 8$. The sharpness of the cliff is a property of the workload imbalance; its existence is a property of the privacy stack composition.

The single recommendation this thesis makes to the privacy-preserving-FL-for-healthcare community: report majority-class baseline and macro-F1 alongside overall accuracy. The privacy/utility tradeoff curve in any imbalanced-data regime is meaningless without them; in our experiments it is the only thing standing between an honest "DP collapsed the model" and a reportable "DP costs 1.7 pp."

The protocol and infrastructure to do so are contributed in `src/` and `experiments/`, all driven by committed YAML configurations. Every number in this document can be reproduced with `python -m experiments.run_*` on the corresponding config.

Future work (§9.5) is multi-seed validation, records 200–234, modular-integer SecAgg, and a MIMIC-IV port.

Bibliography

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data.* AISTATS 2017, pp. 1273–1282.

[2] Martín Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. *Deep Learning with Differential Privacy.* ACM CCS 2016, pp. 308–318.

[3] Ilya Mironov. *Rényi Differential Privacy.* IEEE Computer Security Foundations Symposium (CSF) 2017, pp. 263–275.

[4] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. *Practical Secure Aggregation for Privacy-Preserving Machine Learning.* ACM CCS 2017, pp. 1175–1191.

[5] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, Graham Cormode, and Ilya Mironov. *Opacus: User-Friendly Differential Privacy Library in PyTorch.* NeurIPS Privacy in ML Workshop, 2021.

[6] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. *Flower: A Friendly Federated Learning Research Framework.* arXiv:2007.14390, 2020.

[7] George B. Moody and Roger G. Mark. *The Impact of the MIT-BIH Arrhythmia Database.* IEEE Engineering in Medicine and Biology Magazine, 20(3):45–50, 2001.

[8] Association for the Advancement of Medical Instrumentation. *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms (ANSI/AAMI EC57)*, 1998.

[9] Mohammad Kachuee, Shayan Fazeli, and Majid Sarrafzadeh. *ECG Heartbeat Classification: A Deep Transferable Representation.* IEEE International Conference on Healthcare Informatics (ICHI) 2018, pp. 443–444.

[10] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. *Federated Optimization in Heterogeneous Networks.* MLSys 2020.

[11] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. *Federated Learning: Strategies for Improving Communication Efficiency.* NeurIPS Workshop on Private Multi-Party Machine Learning, 2016.

[12] Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. *Towards a Rigorous Evaluation of Time-series Anomaly Detection.* AAAI 2022, pp. 7194–7201.

[13] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. *Membership Inference Attacks Against Machine Learning Models.* IEEE Symposium on Security and Privacy (S&P) 2017, pp. 3–18.

[14] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, et al. *Advances and Open Problems in Federated Learning.* Foundations and Trends in Machine Learning, 14(1–2):1–210, 2021.

[15] Cynthia Dwork. *Differential Privacy.* ICALP 2006, pp. 1–12.

[16] Sivakanth Gopi, Yin Tat Lee, and Lukas Wutschitz. *Numerical Composition of Differential Privacy.* NeurIPS 2021.

[17] H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. *Learning Differentially Private Recurrent Language Models.* ICLR 2018.

[18] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H. Yang, Farhad Farokhi, Shi Jin, Tony Q. S. Quek, and H. Vincent Poor. *Federated Learning with Differential Privacy: Algorithms and Performance Analysis.* IEEE Transactions on Information Forensics and Security, 15:3454–3469, 2020.

[19] James Henry Bell, Kallista A. Bonawitz, Adrià Gascón, Tancrède Lepoint, and Mariana Raykova. *Secure Single-Server Aggregation with (Poly)Logarithmic Overhead.* ACM CCS 2020, pp. 1253–1269.

[20] Jinhyun So, Chaoyang He, Chien-Sheng Yang, Songze Li, Qian Yu, Ramy E. Ali, Basak Guler, and Salman Avestimehr. *LightSecAgg: A Lightweight and Versatile Design for Secure Aggregation in Federated Learning.* MLSys 2022.

[21] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Anima Anandkumar. *signSGD: Compressed Optimisation for Non-Convex Problems.* ICML 2018, pp. 560–569.

[22] Alham Fikri Aji and Kenneth Heafield. *Sparse Communication for Distributed Gradient Descent.* EMNLP 2017, pp. 440–445.

[23] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletari, Holger R. Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N. Galtier, Bennett A. Landman, Klaus Maier-Hein, et al. *The Future of Digital Health with Federated Learning.* npj Digital Medicine, 3:119, 2020.

[24] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. *Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting.* IEEE Computer Security Foundations Symposium (CSF) 2018, pp. 268–282.

[25] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. *Inverting Gradients — How Easy Is It to Break Privacy in Federated Learning?* NeurIPS 2020.

[26] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. *Exploiting Unintended Feature Leakage in Collaborative Learning.* IEEE Symposium on Security and Privacy (S&P) 2019, pp. 691–706.

[27] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. *Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent.* NeurIPS 2017.

[28] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. *Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates.* ICML 2018, pp. 5650–5659.

Appendix A

Architecture and package layout

This appendix gives the system-level view of the framework: the actors, the per-round data flow as a numbered protocol (re-stated from §5.10 with the actors made explicit), and the package layout of the implementation. It expands on the in-body Method chapter with the level of operational detail a reader who wants to extend the framework will need.

A.1 Actors

```
+---------------------------+         +------------------------------+
|   IoT Healthcare Device   |   ...   |   Aggregation Server         |
|  (simulated client x N)   | <-----> | (orchestrator + aggregator)  |
+---------------------------+         +------------------------------+
        ^                                          ^
        | holds raw ECG (private)                  | holds global model state
        | runs DP-SGD locally                      | runs custom FedAvg strategy
        | masks update with SecAgg                 | unmasks via SecAgg
        | compresses delta                         | decompresses + aggregates
```

IoT client. Owns a partition of patient ECG beats; never shares raw data; trains locally. In the simulation each client is a Python process orchestrated by Flower's Ray-based simulation backend.

Aggregation server. Coordinates rounds; aggregates masked, compressed updates; never sees raw data. In the simulation the server is a single Python process running the custom `PrivacyAwareFedAvg` strategy.

Adversary (modeled). Honest-but-curious server, a coalition of curious clients, or a network eavesdropper. The detailed adversary characterisation is Chapter 4.

A.2 Per-round protocol

The protocol below is the sequence of messages exchanged in one training round. The encryption envelope (TLS) is assumed at the transport layer and not shown.

```
Round t:
  S -> C_i: broadcast w_t                                (for each client i)
  C_i:      run local DP-SGD for E epochs on D_i
  C_i:      Delta_i = w_local - w_t                      (client-side delta)
```

```
C_i:      Delta_i = compress(Delta_i)                    (optional)
C_i:      Delta_i = mask_update(Delta_i, i, session)     (optional, SecAgg)
C_i -> S: send (Delta_i, n_i)
S:        Sum = sum_i Delta_i                            (masks cancel)
S:        Sum = decompress(Sum)                          (optional)
S:        w_{t+1} = w_t + (1 / sum_i n_i) * sum_i n_i * Delta_i
```

The cryptographic property that makes the cancellation step work is that every pairwise mask appears once with + and once with − across the client set; §2.2.2 contains the full algebra. Any *strict subset* of masked updates remains a uniformly random vector, so the server learns nothing about an individual client's update.

A.3 Package layout (`src/`)

```
src/
|-- data/                       # everything that touches raw signals
|   |-- mitbih_loader.py        # downloads MIT-BIH (or synthetic fallback)
|   |-- preprocessing.py        # bandpass filter, beat segmentation, AAMI 5-class
|   `-- partitioner.py          # IID / Dirichlet non-IID splits across N devices
|
|-- models/                     # DL architectures
|   |-- ecg_cnn.py              # full 1D CNN (~50k params)
|   `-- lightweight.py          # tiny variant (~5k params) for edge constraints
|
|-- client/                     # client-side runtime
|   |-- flower_client.py        # NumPyClient (fit / evaluate / get_parameters)
|   |-- dp_trainer.py           # local training loop wrapped by Opacus PrivacyEngine
|   `-- iot_device.py           # simulates compute / bandwidth limits per device
|
|-- privacy/                    # cryptographic + statistical defenses
|   |-- differential_privacy.py # noise calibration; DP accountant (Opacus wrapper)
|   |-- secure_agg.py           # PRG-based pairwise mask primitives
|   `-- compression.py          # top-k sparsification + 8-bit quantization
|
|-- server/                     # server-side runtime
|   |-- flower_server.py        # entry point; starts simulation
|   |-- strategies.py           # PrivacyAwareFedAvg with hooks for SecAgg + decompression
|   `-- secure_aggregation.py   # server's view of the masking protocol
|
`-- utils/                      # cross-cutting helpers
    |-- config.py               # YAML config loader with CLI override support
    |-- metrics.py              # accuracy, macro-F1, per-class P/R
    |-- logging_utils.py        # console + file handlers
    `-- seed.py                 # reproducibility
```

A.4 Configuration knobs

The most consequential YAML keys (full schema in Appendix B):

- `federation.num_clients` — total IoT devices.
- `federation.clients_per_round` — sampled fraction (Flower's `fraction_fit`).

- `federation.local_epochs` — local SGD passes per round.
- `privacy.differential_privacy.target_epsilon`, `target_delta` — DP budget.
- `privacy.differential_privacy.max_grad_norm` — DP gradient clip threshold.
- `privacy.secure_aggregation.enabled` — pairwise masking on/off.
- `privacy.compression.top_k_ratio` — fraction of weights kept per layer.
- `privacy.compression.quant_bits` — quantization width (default 8).

A.5 Where the substance lives

For a reader who wants to inspect the implementation, the four files that carry the bulk of the contribution are:

- `src/server/strategies.py` — the custom `PrivacyAwareFedAvg` that orchestrates the privacy stack.
- `src/privacy/differential_privacy.py` — DP accounting and `PrivacyEngine` attachment (Listing C.3).
- `src/privacy/secure_agg.py` — the pairwise-mask correctness argument in code (Listing C.1).
- `src/client/flower_client.py` — the client side of every round.

Appendix B

Resolved YAML configurations

This appendix lists the five YAML configurations that produce every number in the thesis. The five configurations form an inheritance chain: `default.yaml` is the base, the three single-defence configurations override the relevant `privacy.*` subtree, and `experiment_full.yaml` enables all three. Each `run_*.py` entry point also dumps a `resolved_config.yaml` into the run's `results/` directory; the listings below are the on-disk source configs, not the post-resolution output.

B.1 `configs/default.yaml`

Base configuration. Vanilla FedAvg, no privacy defences enabled. Records 100–115; 4 clients; Dirichlet $\alpha = 0.5$ partitions; 10 rounds; 2 local epochs; SGD with $\eta = 0.01$, momentum 0.9, weight decay 10^{-4}; batch 32.

```yaml
experiment:
  name: vanilla_fedavg
  output_dir: results/vanilla_fedavg
  seed: 42

data:
  source: mitbih          # mitbih | synthetic
  cache_dir: data/cache
  records: [100, 101, 103, 105, 106, 108, 109, 111, 112, 113, 114, 115]
  window_size: 360        # samples per ECG beat window (~1 second @ 360 Hz)
  num_classes: 5          # AAMI 5-class scheme (N, S, V, F, Q)
  test_fraction: 0.2

partitioning:
  scheme: dirichlet       # iid | dirichlet
  dirichlet_alpha: 0.5    # lower = more non-IID
  min_samples_per_client: 50

federation:
  num_clients: 4
  clients_per_round: 4    # all clients participate every round
  num_rounds: 10
  local_epochs: 2
  local_batch_size: 32

model:
```

```yaml
  name: ecg_cnn           # ecg_cnn | lightweight
  dropout: 0.3

optimization:
  optimizer: sgd
  learning_rate: 0.01
  momentum: 0.9
  weight_decay: 0.0001

privacy:
  differential_privacy:
    enabled: false
  secure_aggregation:
    enabled: false
  compression:
    enabled: false

evaluation:
  evaluate_every: 1       # evaluate global model every N rounds
  metrics: [accuracy, macro_f1, per_class_f1]

logging:
  level: INFO
  log_to_file: true
```

B.2 configs/experiment_secagg.yaml

Adds pairwise-mask SecAgg. All other keys inherit from `default.yaml`.

```yaml
experiment:
  name: federated_secagg
  output_dir: results/federated_secagg

privacy:
  differential_privacy:
    enabled: false
  secure_aggregation:
    enabled: true
    seed_size_bytes: 32     # PRG seed size used to derive mask material
    fixed_point_bits: 16    # quantization for fixed-point mask arithmetic
  compression:
    enabled: false
```

B.3 configs/experiment_dp.yaml

Adds DP-SGD via Opacus, target $\varepsilon = 5$, target $\delta = 10^{-5}$. Increases rounds (20) and decreases local epochs (1) and learning rate (0.005); raises batch size to 64 for better gradient signal-to-noise under DP noise.

```yaml
experiment:
  name: federated_dp
```

```yaml
  output_dir: results/federated_dp

privacy:
  differential_privacy:
    enabled: true
    target_epsilon: 5.0
    target_delta: 1.0e-5
    max_grad_norm: 1.0      # gradient clipping threshold C
    noise_multiplier: null  # null = auto-calibrate to hit target_epsilon
    accountant: rdp         # rdp | gdp
  secure_aggregation:
    enabled: false
  compression:
    enabled: false

federation:
  num_rounds: 20
  local_epochs: 1
  local_batch_size: 64       # larger batch -> better signal-to-noise

optimization:
  learning_rate: 0.005
```

B.4 configs/experiment_full.yaml

The headline experiment: DP + SecAgg + top-k/int8 compression. 30 rounds; target $\varepsilon = 8$ (slightly looser than the DP-only run to compensate for compression noise). Compression keeps the top 10% of magnitudes per layer and quantises to int8.

```yaml
experiment:
  name: federated_full_pipeline
  output_dir: results/federated_full_pipeline

privacy:
  differential_privacy:
    enabled: true
    target_epsilon: 8.0      # slightly looser to compensate for compression noise
    target_delta: 1.0e-5
    max_grad_norm: 1.0
    accountant: rdp
  secure_aggregation:
    enabled: true
    seed_size_bytes: 32
    fixed_point_bits: 16
  compression:
    enabled: true
    method: top_k_quantize   # top_k_quantize | quantize_only | top_k_only
    top_k_ratio: 0.1         # keep top 10% of weights by magnitude per layer
    quant_bits: 8

federation:
```

```
  num_rounds: 30
  local_epochs: 1
  local_batch_size: 64

optimization:
  learning_rate: 0.005
```

B.5 Follow-up configs (`configs/followup_records200/`)

The records 200–234 follow-up uses the same five configurations with two changes: `data.records` is set to `[200, 201, ..., 234]` and the `output_dir` is rooted at `results/followup_records200/`. No hyperparameters are retuned; the contribution is the relative shape across the privacy stack, not the absolute accuracy on either slice (§8.5).

Appendix C

Selected source code

This appendix reproduces the three short, consequential pieces of source code referenced throughout the thesis: the SecAgg client-side mask and server-side aggregation; the top-k + int8 compression round-trip; and the DP attachment to a PyTorch model + optimiser via Opacus. The full source is available at https://github.com/mapeke/privacy-preserving-fl-healthcare-iot.

C.1 Pairwise-mask SecAgg (`src/privacy/secure_agg.py`)

The two functions below implement Algorithm 2. The asymmetric `if client_id < j: masked += mask else: masked -= mask` is the entire cancellation property in code form. The same function is exercised by `tests/test_secagg.py::test_masks_cancel_under_aggregation`.

```
def derive_mask(seed: bytes, shape) -> np.ndarray:
    """Expand a pair seed into a Gaussian mask vector via NumPy's PCG64."""
    if isinstance(shape, int):
        shape = (shape,)
    seed_int = int.from_bytes(seed, byteorder="big", signed=False)
    rng = np.random.default_rng(seed_int)
    return rng.standard_normal(size=shape).astype(np.float32)


def mask_update(update, client_id: int, session: SecAggSession) -> np.ndarray:
    """Add the pairwise masks for client_id to update.

    For every other client j: if client_id < j, add +m_{client_id,j};
    otherwise add -m_{j,client_id}. Summing across all clients cancels
    every mask exactly.
    """
    if not (0 <= client_id < session.num_clients):
        raise ValueError(f"client_id {client_id} out of range")

    masked = update.astype(np.float32, copy=True)
    for j in range(session.num_clients):
        if j == client_id:
            continue
        seed = session.seed_for(client_id, j)
        mask = derive_mask(seed, update.shape)
        if client_id < j:
            masked += mask
```

```
        else:
            masked -= mask
    return masked


def aggregate_masked_updates(masked_updates) -> np.ndarray:
    """Element-wise sum; pair-masks cancel by construction."""
    if not masked_updates:
        raise ValueError("empty update list")
    stacked = np.stack(masked_updates, axis=0)
    return stacked.sum(axis=0)
```

C.2 Top-k + int8 compression (src/privacy/compression.py)

`top_k_sparsify` uses `np.argpartition` for $O(n)$ top-k selection rather than the $O(n \log n)$ full sort. `quantize_int8` computes the `(scale, zero_point)` pair from the per-tensor min/max so that dequantisation reconstructs the floating-point values to within ± 1 LSB.

```
def top_k_sparsify(update, ratio: float):
    """Keep only the top-ratio entries by absolute value (O(n))."""
    if not 0.0 < ratio <= 1.0:
        raise ValueError("ratio must be in (0, 1]")
    flat = update.ravel().astype(np.float32, copy=False)
    k = max(1, int(round(flat.size * ratio)))
    if k >= flat.size:
        return np.arange(flat.size, dtype=np.int32), flat.copy()
    idx = np.argpartition(np.abs(flat), -k)[-k:]
    idx = np.sort(idx).astype(np.int32)
    return idx, flat[idx].copy()


def quantize_int8(values):
    """Min-max quantise a float vector into int8.

    Returns (int8_values, scale, zero_point) so that
        values ~= scale * (int8_values.astype(float32) - zero_point)
    """
    if values.size == 0:
        return values.astype(np.int8), 1.0, 0.0
    v_min = float(values.min())
    v_max = float(values.max())
    if v_max == v_min:
        return np.zeros_like(values, dtype=np.int8), 1.0, 0.0
    qmin, qmax = -128, 127
    scale = (v_max - v_min) / (qmax - qmin)
    zero_point = qmin - v_min / scale
    quant = np.round(values / scale + zero_point)
    quant = np.clip(quant, qmin, qmax).astype(np.int8)
    return quant, float(scale), float(zero_point)
```

```
def dequantize_int8(quant, scale: float, zero_point: float):
    return (quant.astype(np.float32) - zero_point) * scale
```

C.3 DP attachment (src/privacy/differential_privacy.py)

attach_dp is a thin wrapper around Opacus's PrivacyEngine.make_private_with_epsilon. The function-level import of Opacus is the lazy-import invariant tested in tests/test_dp.py: the rest of the test suite must run without Opacus installed.

```
@dataclass
class DPConfig:
    target_epsilon: float = 5.0
    target_delta: float = 1e-5
    max_grad_norm: float = 1.0
    noise_multiplier: float | None = None  # None => auto-calibrate
    accountant: str = "rdp"                # "rdp" | "gdp" | "prv"

    def validate(self) -> None:
        if self.target_epsilon <= 0:
            raise ValueError("target_epsilon must be positive")
        if not (0 < self.target_delta < 1):
            raise ValueError("target_delta must be in (0, 1)")
        if self.max_grad_norm <= 0:
            raise ValueError("max_grad_norm must be positive")


def attach_dp(model, optimizer, data_loader, config: DPConfig, epochs: int):
    """Wrap (model, optimizer, data_loader) with Opacus PrivacyEngine.

    If config.noise_multiplier is None, Opacus solves for sigma to hit
    config.target_epsilon over ‘epochs‘ epochs of the given dataloader.
    """
    config.validate()
    from opacus import PrivacyEngine          # lazy import (CI-asserted)

    engine = PrivacyEngine(accountant=config.accountant)
    if config.noise_multiplier is None:
        model_p, optimizer_p, loader_p = engine.make_private_with_epsilon(
            module=model,
            optimizer=optimizer,
            data_loader=data_loader,
            target_epsilon=config.target_epsilon,
            target_delta=config.target_delta,
            max_grad_norm=config.max_grad_norm,
            epochs=epochs,
        )
    else:
        model_p, optimizer_p, loader_p = engine.make_private(
            module=model,
            optimizer=optimizer,
            data_loader=data_loader,
```

```
        noise_multiplier=config.noise_multiplier,
        max_grad_norm=config.max_grad_norm,
    )
return DPHandles(
    engine=engine,
    model=model_p,
    optimizer=optimizer_p,
    data_loader=loader_p,
    epochs=epochs,
)
```

Appendix D

Reproducibility checklist

This appendix lists the exact steps to reproduce every number in the thesis from a clean clone. No tuning is required; every reported result is tied to one committed YAML configuration.

D.1 Environment

- **OS:** Windows 10 Pro (the development machine); equivalent Linux distributions work without modification.
- **Python:** 3.12 (3.10+ is sufficient).
- **GPU:** a single CUDA-capable GPU. Development was on a mobile-class NVIDIA RTX with 6 GB VRAM; the framework is not memory-bound and runs on smaller GPUs without issue.
- **CUDA:** 12.1 (matched by PyTorch 2.4 wheel).
- **Disk:** ∼200 MB for the MIT-BIH cache, ∼50 MB for run artefacts.

D.2 Setup

```
git clone https://github.com/mapeke/privacy-preserving-fl-healthcare-iot.git
cd privacy-preserving-fl-healthcare-iot
pip install -r requirements.txt
pytest tests/ -q       # 42 passed in ~4 s
```

D.3 Reproducing the five primary experiments (records 100–115)

Each command reads its config from `configs/`, writes its results into `results/<name>/`, and dumps a `resolved_config.yaml` so the exact configuration of the run is on disk alongside the metrics.

```
PYTHONPATH=. python -m experiments.run_centralized      \
    --config configs/default.yaml --epochs 5
PYTHONPATH=. python -m experiments.run_federated        \
    --config configs/default.yaml
PYTHONPATH=. python -m experiments.run_federated_dp     \
    --config configs/experiment_dp.yaml
PYTHONPATH=. python -m experiments.run_federated_secagg \
    --config configs/experiment_secagg.yaml
```

```
PYTHONPATH=. python -m experiments.run_full_pipeline    \
    --config configs/experiment_full.yaml
```

D.4 Reproducing the records 200–234 follow-up

Same five entry points, with the four follow-up configs:

```
PYTHONPATH=. python -m experiments.run_federated        \
    --config configs/followup_records200/default.yaml
PYTHONPATH=. python -m experiments.run_federated_secagg \
    --config configs/followup_records200/experiment_secagg.yaml
PYTHONPATH=. python -m experiments.run_federated_dp     \
    --config configs/followup_records200/experiment_dp.yaml
PYTHONPATH=. python -m experiments.run_full_pipeline    \
    --config configs/followup_records200/experiment_full.yaml
```

D.5 Approximate run-times (single RTX-class GPU)

Run	Wall time
Centralized (5 epochs)	∼1 min
Vanilla FedAvg (10 rounds)	∼3 min
FedAvg + SecAgg (10 rounds)	∼3 min
FedAvg + DP (20 rounds)	∼30 min
Full pipeline (30 rounds)	∼30 min
Records 200–234 follow-up (all four runs)	∼70 min

D.6 What is logged per run

Each `results/<name>/` directory contains:

- `run.log` — the full console log of the run (round-by-round accuracy, ε updates, any warnings).
- `resolved_config.yaml` — the exact configuration applied, including any CLI overrides.
- `history.json` — the Flower-style metrics history, parsed by the analysis notebooks.

D.7 Single seed; multi-seed is future work

Every reported number is a single-seed run at `seed: 42` (set in `configs/default.yaml`). Multi-seed validation is the first item of §9.5; it would put error bars on Table 6.1 and on the records 200–234 follow-up table. The DP class-prior collapse (Finding 6.1) is structural rather than seed-dependent — it is the model converging to the majority-class predictor, which the same training process produces from any random initialisation — so multi-seed would not change Finding 6.1 qualitatively.